DKV: Anchor + Low-Rank Differential KV-Cache Compression for Scalable Long-Context Inference

Om Chimurkar

Newton School of Technology, Rishihood University

omchimurkar45@gmail.com · https://github.com/Omc12/Differential-KV

Abstract

The key–value (KV) cache is the memory wall of long-context transformer inference: its size grows linearly with sequence length, and on commodity hardware it, not the model weights, is what first exhausts memory. We present DKV, a KV-cache compression runtime that keeps recent tokens exact and compresses older ones. Each block of B=256 tokens is reduced to an *anchor* token (kept exact), a rank-r joint $K|V$ truncated-SVD *delta* that captures the block's shared structure, and a budget of R *exact residual tokens*—the rows the low-rank basis reconstructs worst, which are precisely the distinctive tokens (digits, names, codes) that verbatim recall depends on. A bounded dense recency window holds the newest tokens uncompressed. At decode time a fused decode buffer routes to the top-K relevant blocks, scores the query against them *in low-rank space without ever decompressing* K, attends the residuals and recency window exactly, and merges the two halves with a flash-style log-sum-exp reduction; the selected blocks are materialised once per routing interval into a persistent buffer and appended to per token. Prefill uses a training-free block-sparse attention so its cost grows sub-quadratically.

We implement DKV entirely in MLX and evaluate it on an Apple M3 with 8.6 GB of unified memory against two dense full-KV baselines: a memory-optimized `mlx_lm` engine sharing DKV's exact Qwen2.5-1.5B int4 weights—the controlled comparison—and a standard PyTorch engine on the unquantized fp16 checkpoint, included as the configuration a practitioner gets by default rather than as a weight-matched control.

DKV holds a bounded KV state that is 1.44×–2.25× smaller per block than a dense cache (the residual budget R is the knob), recovers a buried passcode *exactly* at every context from 4k to 64k, and reaches 64k—where the default PyTorch full-KV configuration runs out of memory at 16k on the same host, and which even the memory-optimized dense cache reaches only by prefilling 1.72× slower than DKV. The cost is decode throughput: reconstructing and merging the sparse representation makes per-token decode slower than a dense cache while the dense cache still fits. We report this trade-off in full, isolate each component with ablations, and characterize the residual budget as an explicit accuracy–memory–speed dial. All numbers in this report are measured on the described host; none are estimated.

A parallel CUDA/Triton engine—a custom Triton fused decode kernel with `@triton.autotune` tiling, asynchronous H2D block prefetch via dedicated `cudaStreams`, tiered CPU–HBM block offloading informed by the kTransformers inference framework, and a native C++/GGML decode kernel built with `-DGGML_CUDA=ON`—is implemented and CPU-verified. No GPU benchmark numbers are collected in this report and no GPU performance claims are made; the tables reserve space for them without restructuring.

1. Introduction

Autoregressive transformer inference caches the key and value projections of every past token so that each new token attends over the whole context without recomputation. This KV cache is what makes decoding fast, but it is also what makes long context expensive: its size grows linearly with sequence length and with the number of layers, and for a small model on commodity hardware it overtakes the weights as the dominant consumer of memory. For Qwen2.5-1.5B the int4 weights occupy roughly a gigabyte, while a full fp16 KV cache costs about 28 KB per token across all layers—so a 64k-token conversation asks for close to 1.8 GB of cache on top of the model, on a machine that may have only 8 GB of unified memory shared with the operating system, the display server, and every other process. The cache, not the model, is the wall.

A large body of work attacks this wall by making the cache smaller or sparser: low-bit quantization of K/V, eviction of tokens deemed unimportant, and low-rank factorization of the cache tensors. Each trades some fidelity for memory. The failure mode that matters in practice is not average perplexity but *verbatim recall*: a summary that paraphrases the document is often acceptable, but a summary that reports the wrong passcode, account number, or name is not. Aggressive compression tends to destroy exactly the low-frequency, high-information

tokens on which such recall depends, because those tokens are outliers that a shared low-rank or low-bit representation reconstructs worst.

DKV is a KV-cache compression runtime built around this observation. It keeps a bounded window of recent tokens exact, and compresses each older block of B=256 tokens into three parts: an *anchor* (the block's first token, kept exact and used as the baseline for the rest), a rank-r joint $K \mid V$ truncated-SVD *delta* that captures the structure the block's tokens share, and a small budget of *exact residual tokens*—selected by a multi-signal ranking: reconstruction error identifies the outliers the low-rank basis fails to represent; owner-capture preserves the entity names that own those values; edge-capture rescues relational connectives whose reconstruction collides with a neighbour's key. The anchor-plus-delta captures the smooth, shared part of the block cheaply; the residuals rescue the outliers exactly. Decode never materializes the full cache: a fused buffer scores the query against each block in the rank-r subspace, attends the exact residuals and the recency window directly, and combines the two contributions with a numerically stable log-sum-exp merge. A cheap per-block router restricts the expensive part of this computation to the top-K relevant blocks, so decode cost scales with K rather than with the context length.

What this paper claims, and what it does not. We are deliberate about the trade-off, because it is easy to overstate. On our host—an Apple M3 with 8.6 GB of unified memory, running Qwen2.5-1.5B at int4—an optimized dense full-KV cache *fits and decodes faster than* DKV through 32k tokens; DKV does not make this model decode faster where a dense cache still fits. But memory tells the other half of the story: the default PyTorch full-KV configuration exhausts the machine at 16k and does not run beyond it (on fp16 weights—see §8 for why that engine is a practitioner reference rather than a weight-matched control), and even the optimized dense—which shares DKV's int4 weights and does reach 64k—gets there only by prefilling 1.72× slower than DKV, whose bounded store completes with the passcode intact. What DKV provides is therefore (i) a KV state that is bounded by a fixed block pool and is 1.44×–2.25× smaller per block than a dense cache, so the cache stops being the term that grows without limit and 64k becomes reachable where a naive full-KV cache cannot; (ii) *exact* recovery of a buried passcode at every context we tested, from 4k to 64k, preserved by the residual mechanism; and (iii) a sparse prefill whose sub-quadratic cost lets long-context prefill overtake the dense baseline once the context is large enough. The price is per-token decode throughput *in the regime where a dense cache still fits*. We report the full picture—including where DKV loses—and we treat the residual budget R as an explicit dial between memory, accuracy, and speed.

Contributions.

- An **anchor + low-rank + exact-residual** block representation (§4) that decouples the smooth, shared part of a KV block (compressed) from its distinctive outlier tokens (kept exact), giving a tunable compression ratio while preserving verbatim recall. Residual selection is multi-signal: reconstruction error, entity name preservation (owner-capture), and relational-connective rescue (edge-capture).
- A **fused routed decode kernel with persistent buffer** (§6) that scores queries in the low-rank subspace without decompressing K, attends residuals and recency exactly, merges the halves with a flash-style reduction, and amortises block materialisation across a configurable interval—so decode cost scales with K, not context length.
- A **bounded-memory runtime** (§5) with streaming prefill compression, a fixed block pool, a prefill-to-decode memory release, and a training-free block-sparse prefill, implemented entirely in MLX for Apple unified memory.
- A **measured, self-consistent evaluation** (§8) against an optimized `mlx_lm` dense baseline on *identical* int4 weights—plus a standard PyTorch dense engine as a practitioner-default reference—reporting footprint, prefill, decode, and exact recall from 4k to 64k, with component ablations and a residual-budget sweep. We state the decode-throughput cost, and the limits of each baseline, plainly rather than hiding them.
- **Runtime extensions for GPU-native serving** (§7), informed by the kTransformers framework [18]: tiered CPU–GPU block offloading with heat-scored eviction, step-ahead asynchronous H2D prefetch, autotuned Triton decode kernels [15], and an optional MLA-style latent projection [4]. The CUDA/Triton engine exposes two decode paths: a Triton fused kernel (`_fused_sparse_decode_kernel`, with `@triton.autotune` over `(S_MAX,BLOCKS_PER_CHUNK, num_warps)`) for PyTorch/CUDA environments, and a native C++/GGML kernel (`dkv_full_decode_kernel` in `dkv_decode.cu`, built with `-DGGML_CUDA=ON`) for embedded serving. Both implement the same anchor-plus-delta-plus-residual decode algorithm described in §6. All four runtime optimizations are implemented and CPU-verified; CUDA hardware validation is reserved for a follow-up *(GPU validation pending)*.

The rest of the paper follows the system's own execution order: background and the memory arithmetic that motivates the design (§2), a component overview (§3), the compression pipeline (§4), the runtime and memory architecture (§5), the fused decode kernel (§6), implementation notes (§7), evaluation (§8), analysis (§9), and limitations (§10).

2. Background and Motivation

2.1. The KV-cache memory arithmetic

Consider a decoder-only transformer [16] with L layers, H_{kv} key/value heads (grouped-query attention [1] shares each KV head across several query heads), and head dimension d. Caching one token's keys *and* values across all layers, in fp16, costs

$$b_{\text{tok}} = 2 \cdot L \cdot H_{\text{kv}} \cdot d \cdot 2 \text{ bytes}. \quad (1)$$

For our evaluation model (Qwen2.5-1.5B: L=28, H_{kv}=2, d=128) this is 28,672 bytes per token—about 28 KB. The cache for a sequence of length N is therefore $N \cdot b_{\text{tok}}$ and grows without bound in N: 16k tokens cost 0.47 GB, 32k cost 0.94 GB, and 64k cost 1.8 GB (Eq. (1) is also how we report the dense engines' KV footprint in §8). Against 8.6 GB of unified memory shared with the weights (≈ 1 GB at int4), the OS, and transient prefill activations, the cache is the term that decides whether a long context fits.

2.2. The design space

Three broad families shrink the cache. *Quantization* stores K/V at lower precision (e.g. int4/int8, per-channel or per-token scales) [12]; it is simple and preserves every token, but uniformly low precision erodes the outlier tokens most. *Eviction / selection* keeps only a subset of tokens—attention sinks [17], heavy hitters [19], persistence-of-importance [11], prompt-aware selection [10], or query-aware block retrieval [14]; it can be very compact but risks discarding a token that a later query needed. *Low-rank factorization* exploits redundancy across tokens or channels, replacing the cache with a small basis [4, 5]; it compresses the shared structure well but, by construction, discards the low-energy directions in which distinctive tokens live. Orthogonally, KV *memory management* such as paged attention [9] reduces fragmentation without shrinking the logical cache, and heterogeneous CPU–GPU inference frameworks such as kTransformers [18] keep hot operators on the GPU while offloading cold operators and KV state to CPU memory, extending effective capacity at the cost of host–device transfer latency. DeepSeek-V2's Multi-head Latent Attention (MLA) [4] takes a different angle: it re-trains the model to project keys and values into a shared low-rank latent space at the architecture level, reducing KV cache footprint without post-hoc compression. DKV draws on both ideas: it adapts kTransformers' tiered CPU–GPU offloading to the compressed-block level (§7), and incorporates an optional MLA-style latent projection as a pre-SVD step (§7), without requiring model re-training.

The tension common to all three is that the tokens most expensive to keep—rare, high-entropy tokens such as identifiers, digits, and names—are exactly the tokens that verbatim recall needs, and exactly the tokens a shared low-rank or low-bit code represents worst. A method that only compresses the shared structure will paraphrase fluently and misquote the one fact that mattered.

2.3. Design principle: separate the smooth part from the outliers

DKV responds to that tension directly by splitting each block into two representations with complementary strengths. The bulk of a block—the smooth, correlated variation shared by its tokens—is captured by an anchor and a rank-r joint $K \mid V$ delta, which is cheap and reconstructs the shared structure at low error. The residual outliers—the handful of rows the low-rank basis reconstructs worst—are kept *exact*. Because the residuals are chosen by measured reconstruction error rather than by a hand-picked token class, the mechanism is content- and model-agnostic: it keeps whatever a given block's distinctive tokens happen to be. The recent past is not compressed at all; it lives in a small exact recency window, so short-range attention is always faithful. §8 shows that this split is what lets a buried passcode survive compression exactly (E5) while the low-rank part keeps the per-block budget small.

2.4. Unified memory and MLX

Our target is Apple silicon, where the CPU and GPU share one physical memory pool. This changes the accounting in two ways that shape the implementation. First, there is no host–device copy: a tensor produced on the GPU is visible to NumPy without a transfer, so the randomized SVD used during compression can run in NumPy on the CPU while the attention kernels run on the GPU, with no marshalling cost. Second, "freeing" memory means returning it to a single shared pool that the rest of the system is contending for, so bounding *peak* allocation matters as much as bounding steady state. DKV is built on MLX: it monkey-patches the attention of an `mlx_lm` model, keeps its compressed store in MLX fp16 arrays, uses a persistent fused decode buffer, and explicitly releases the prefill cache at the prefill→decode boundary (§5).

2.5. Discrete GPU memory model

On discrete-GPU hardware (NVIDIA CUDA) the memory model differs from unified memory in two ways that directly shape implementation choices (Figure 1).

Separate HBM pool. The GPU's High-Bandwidth Memory (HBM) is a distinct physical pool from host DRAM. Any tensor crossing the boundary incurs an explicit PCIe host–device (H2D) or device–host (D2H) transfer. For DKV's block-level CPU–GPU offloading, a default block (U: 63×32 int8; V_K, V_V: 2×32×H_{kv}×d fp16; anchors/residuals: fp16) totals ≈ 137 KB, costing ≈ 15 μs

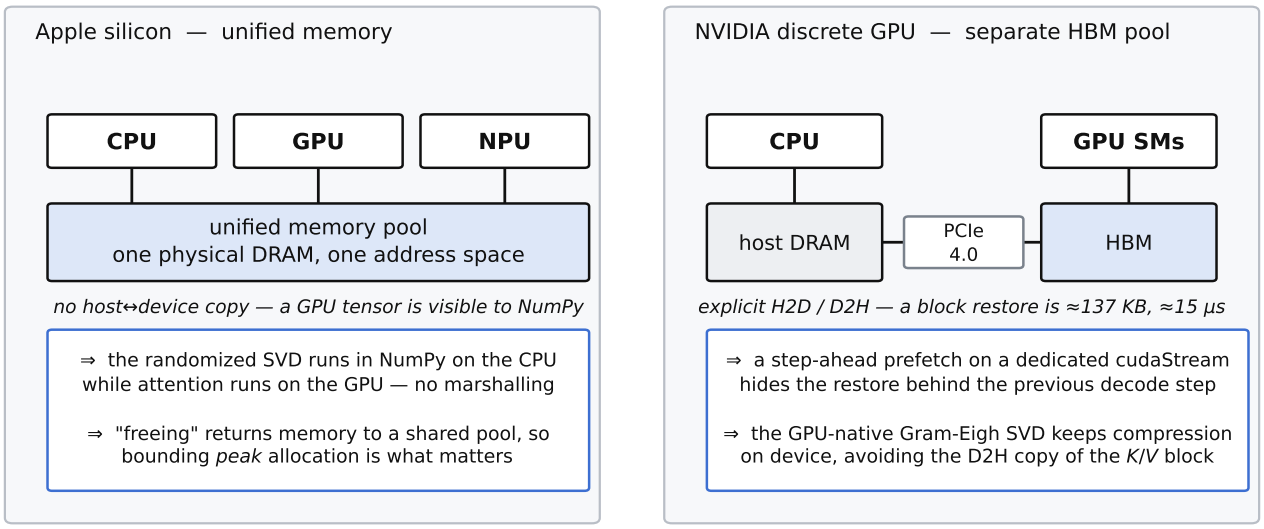

Figure 1 | Memory topology, and what each one implies for DKV. *Left:* on Apple silicon the CPU, GPU and NPU address one physical DRAM pool, so a tensor crosses no boundary—the randomized SVD can run in NumPy on the CPU while attention runs on the GPU, and the binding constraint is *peak* allocation against a pool the whole system shares. *Right:* on a discrete GPU, HBM is a separate physical pool behind PCIe, so every block restore is an explicit H2D transfer ($\approx$ 137 KB, $\approx$ 15 μs). That single difference is what motivates the step-ahead async prefetch and the GPU-native Gram-Eigh SVD (§7.1).

on PCIe 4.0. At decode cadences of 20–80 ms per token this is below 1% overhead even for a cold hit—and the step-ahead async prefetch (§7) hides it entirely by issuing H2D restores a step ahead via a dedicated `cudaStream`.

Compression on a discrete GPU. On MLX the randomized SVD runs in NumPy on the CPU at near-zero marshalling cost (unified memory). On a discrete GPU, the same CPU SVD is still available but incurs a D2H copy of the raw K/V block at compression time—acceptable as a one-time prefill cost per block. A GPU-native path uses the *Gram-Eigh reformulation*: computing the $r\times r$ symmetric Gram matrix $BB^\top$ and calling `cuSOLVER` `syevjBatched`, which batches correctly for $r\leq 32$ (unlike `gesvdjBatched`, which falls back to a sequential loop for matrices larger than 32×32). Controlled CPU measurements show this reduces compression time from $\approx$ 6.1 s to $\approx$ 2.6 s per representative prefill; GPU recall validation is pending *(GPU validation pending)* (§7).

3. System Overview

DKV is structured as a thin serving stack over an MLX inference core whose attention has been replaced (Figure 2). We walk the components in the order data flows through them.

Serving. An OpenAI-compatible gateway and a continuous-batching decode engine accept requests; a session manager owns per-conversation lifecycle and residency. These layers are standard and are not the subject of this paper; they exist so that the compression runtime is exercised under a realistic request path.

Wrapper and model. `MLXDKVWrapper` loads an `mlx_lm` model, patches its attention, and exposes `generate()`, which tokenizes the prompt, runs prefill in fixed-size chunks, and then decodes one token at a time. `MLXQwenModel` wraps the patched model, holds a native MLX KV cache used only during prefill, and—critically—*releases* that prefill cache at the prefill→decode boundary so that decode-time memory reflects the compressed store rather than a retained full-context cache (§5).

Patched attention. The heart of the system is a replacement for the model's attention `__call__`. It branches on sequence length. During prefill (L>1) it computes causal attention—optionally block-sparse (§5)—over the growing native cache to produce correct hidden states, and captures each chunk's K/V into the store. During decode (L=1) it ingests the newly generated token's K/V into the dense window and computes the attention output from the compressed store via the fused decode buffer (§6).

The store. `MLXKVBlockManager` holds all persistent state, per session and per layer: a *dense recency window* of the most recent W=1,024 tokens in exact fp16, and a *compressed block pool* of up to M=256 blocks, each holding the anchor, low-rank factors, exact residuals, and router statistics for one 256-token span. The pool is pre-

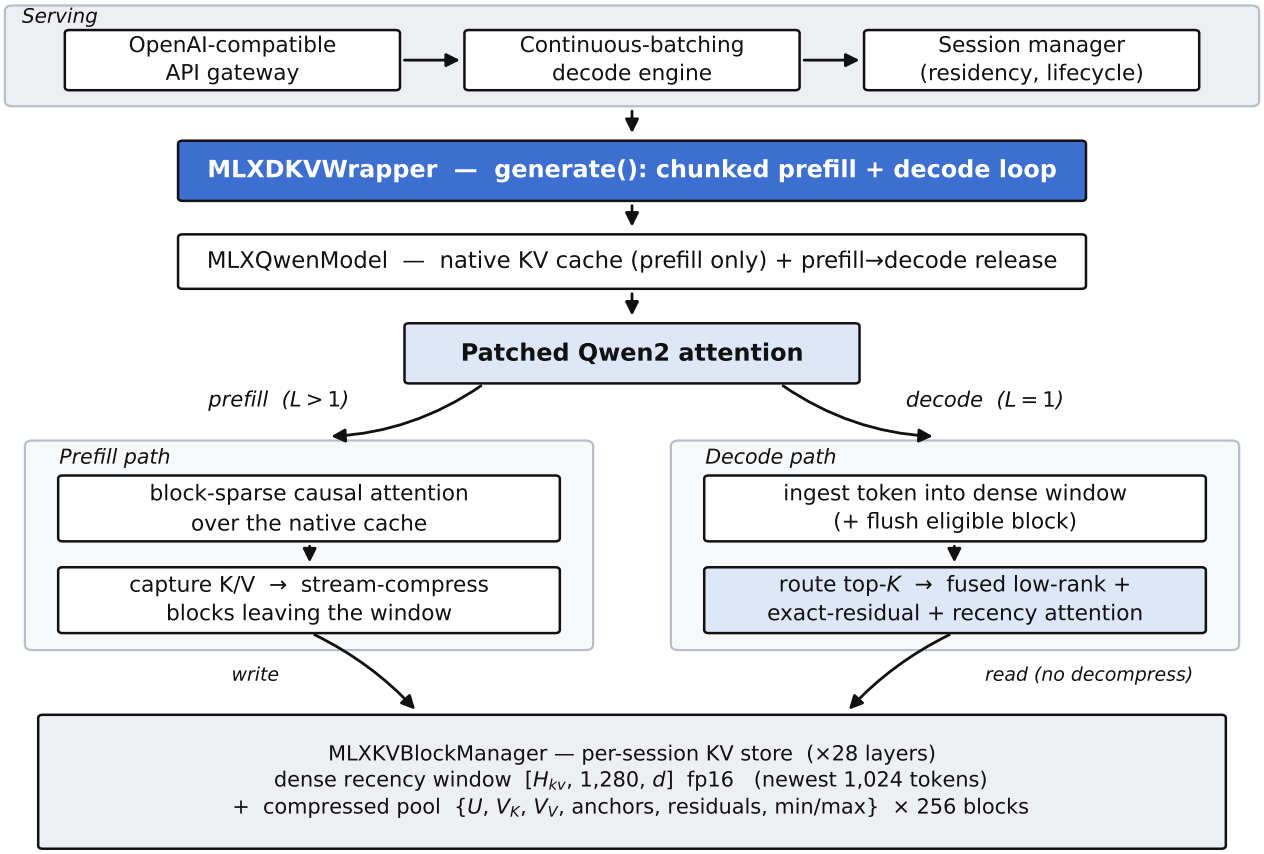

Figure 2 | DKV active-runtime architecture. A serving stack (gateway, batching engine, session manager) drives `MLXDKVWrapper`, which runs chunked prefill and a greedy or sampled decode loop over `MLXQwenModel`. The model's attention layer is monkey-patched: on prefill (L>1) it runs (block-sparse) causal attention over the native cache and captures K/V into the store; on decode (L=1) it ingests the current token and runs the fused routed sparse attention. All state lives in the per-session `MLXKVBlockManager`: a dense recency window plus a bounded compressed block pool.

allocated and fixed in size, which is what bounds the runtime's memory (§5, Figure 4).

Two data paths, one store. Prefill *fills* the store (capture, then stream-compress the oldest blocks as the window overflows); decode *reads* it (route, reconstruct in low-rank space, attend residuals and window). The compression pipeline (§4) defines the store's contents; the runtime (§5) defines its lifecycle; the decode kernel (§6) defines how it is read. We take these in turn.

Scope. This report documents the *active runtime*—the MLX implementation, which is what Figure 2 depicts. Two independent backends also exist in the repository and are out of scope for evaluation: a CUDA/Triton engine (`native_triton_sparse_attn_decode()`, dispatching to `_fused_sparse_decode_kernel` or a PyTorch vectorized fallback) and a native C++/GGML engine (`execute_cuda_attention()` to `dkv_full_decode_kernel` in `dkv_decode.cu`, built with `-DGGML_CUDA=ON`). Both share the identical block store format and compression pipeline described in §4; Figure 8 places all three side by side, and porting the evaluation is a hardware availability problem, not an algorithmic redesign. Where the decode design ports cleanly we say so and reserve space for future measurement (§6, §7, §10). A separate factual-store / semantic-retrieval subsystem is present but disabled by default and is not part of the evaluated system.

4. Differential KV Compression

A block is B=256 consecutive tokens' keys and values, $K, V \in \mathbb{R}^{H_{kv} \times B \times d}$. Compression (Figure 3) runs when a block leaves the recency window. It produces four things: an anchor, a low-rank delta basis, a set of exact residual tokens, and per-block router statistics.

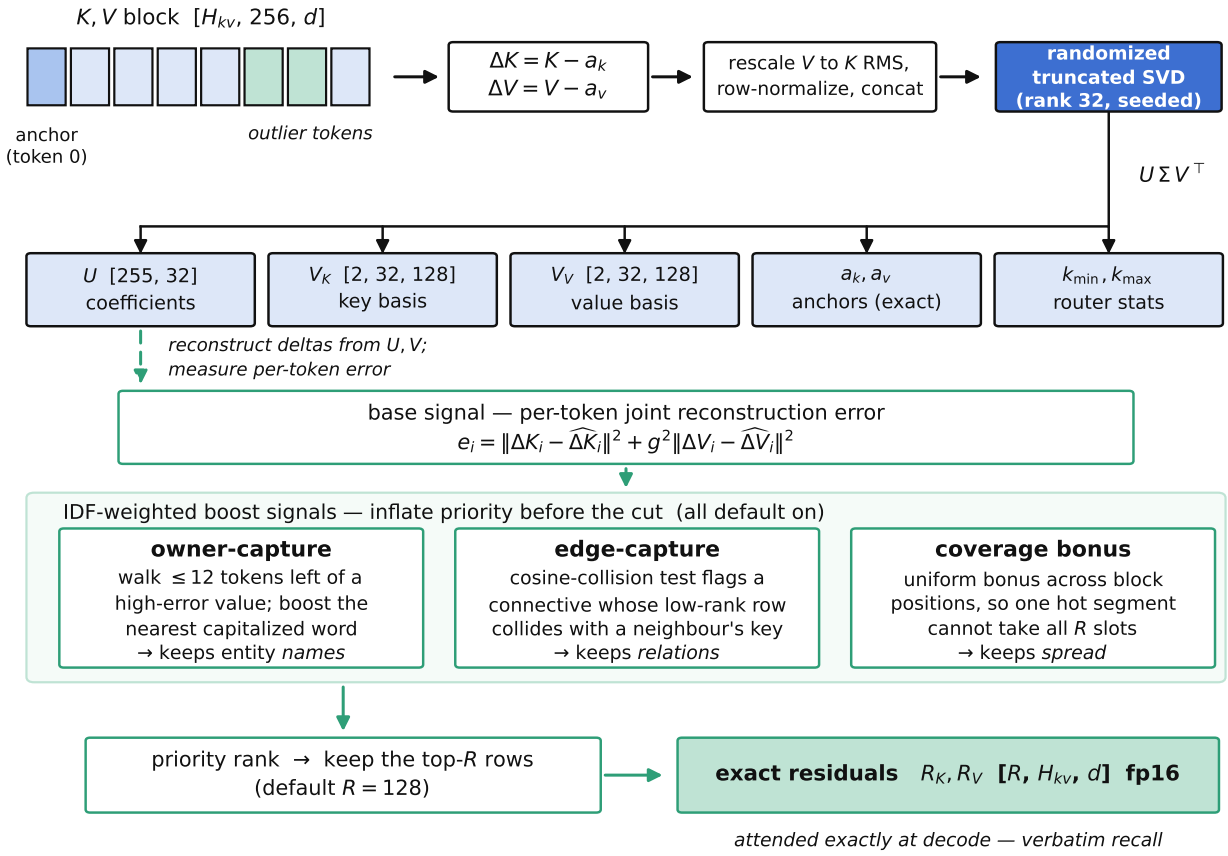

Figure 3 | Compression of one 256-token block. The anchor (token 0) is subtracted to form deltas; V is rescaled to K's RMS and the rows are L_2-normalized before a seeded randomized truncated SVD produces U, V_K, V_V. In parallel, per-token joint reconstruction error forms the *base* selection signal, which three IDF-weighted boost signals (owner-capture, edge-capture, coverage) inflate before the top-R cut; the surviving rows are kept as *exact* residuals R_K, R_V. Key min/max are retained for the decode router. At the default R=128 a block stores ≈ 177.9 KiB vs. 256 KiB dense (1.44×); the R=64 preset stores ≈ 113.9 KiB (2.25×).

4.1. Anchor and delta

Token 0 of the block is the *anchor* $a_k, a_v \in \mathbb{R}^{H_{kv} \times d}$, stored exactly. Every other token is represented relative to it:

$$\Delta K_i = K_i - a_k, \qquad \Delta V_i = V_i - a_v, \qquad i = 1, \ldots, B-1. \quad (2)$$

Subtracting a within-block baseline concentrates the energy that the low-rank basis must capture, so a small rank suffices for the shared structure.

4.2. Joint $K|V$ low-rank factorization

K and V are compressed *jointly*: for each token we stack $[\Delta K_i \,\|\, \Delta V_i]$ and factor the $(B-1) \times 2H_{kv}d$ delta matrix once, so a single set of coefficients U drives both reconstructions. Two normalizations make the joint factorization well-posed. In Qwen2.5 the key norms dominate the value norms, so before factoring we rescale V's deltas to K's RMS energy via a per-block gain

$$g = \mathrm{clip}\left(\sqrt{\frac{\|\Delta K\|_F^2}{\|\Delta V\|_F^2}},\ 1,\ 10^4\right), \quad (3)$$

which is then undone on the stored V_V. The lower clamp ($g \geq 1$) prevents artificial de-weighting of V on well-balanced blocks; the upper clamp ($g \leq 10^4$) guards degenerate blocks where $\|\Delta V\|_F \approx 0$. Without this rescaling the SVD spends its budget on K and the V reconstruction is poor. We then L_2-normalize each token's stacked delta row so the factorization is not dominated by a few high-norm tokens, folding the norms back into U afterwards.

The factorization is a *randomized truncated SVD* [6] with a fixed seed (default 1234): a Gaussian range finder with power iterations, a QR step, and a small dense SVD of the projected matrix, all in fp32 in NumPy (§7 explains why NumPy). The rank is adaptive: it keeps the smallest number of components (at least 4, at most r) that explain

99.9% of the delta energy. The result, cast to fp16, is

$$U \in \mathbb{R}^{(B-1)\times r}, \quad V_K, V_V \in \mathbb{R}^{H_{\text{kv}}\times r\times d}, \tag{4}$$

so that the implicit reconstruction of any non-anchor token is

$$\hat{K}_i = a_k + s\,(U_i V_K), \qquad \hat{V}_i = a_v + s\,(U_i V_V), \tag{5}$$

with s the stored per-block scale. Crucially, decode never forms $\hat{K}_i$ explicitly (§6); Eq. (5) is the semantics, not the runtime path.

4.3. Exact residuals: selecting the outliers

The low-rank basis reconstructs the shared structure well but, by construction, mis-reconstructs low-energy outliers—which are the distinctive tokens recall depends on. DKV measures this directly. For every token it computes the joint reconstruction error $e_i = \left\|\Delta K_i - \widehat{\Delta K}_i\right\|^2 + g^2\left\|\Delta V_i - \widehat{\Delta V}_i\right\|^2$ (the V term reweighted by the same gain g from Eq. (3), so both halves are on a comparable scale when ranking outliers).

Selection is *multi-signal*: reconstruction error is the base signal, but three IDF-weighted boost signals inflate certain tokens' priority before the top-R cut. **Owner-capture** (default on): for each high-error segment, the algorithm walks left up to 12 tokens and boosts the nearest capitalized word into the residual set alongside its value—preserving entity *names* that are low-energy prose yet essential for attribute-binding queries. **Edge-capture** (default on): relational connectives ("grows to", "needs X because Y") are low-energy prose whose low-rank reconstruction collides with a neighbouring token's key; a direct cosine-collision test detects this and boosts such tokens before the residual cut, so the model cannot substitute a neighbour's relation. **Coverage bonus** (default on): a uniform bonus across block positions prevents a single high-error segment from monopolising all R slots, spreading exact coverage across the block.

After applying boosts, the R highest-priority tokens are kept as *exact* fp16 residuals $R_K, R_V \in \mathbb{R}^{R\times H_{\text{kv}}\times d}$. Their low-rank twins are excluded from the reconstruction pool so each such token is represented once, exactly (§6). Selecting by measured error and collision signal, rather than by a token-type heuristic, is what makes the residual mechanism content- and model-agnostic. We validate empirically (§8, E5–E6) that (i) residuals are what make a buried passcode survive compression exactly, and (ii) the residual reconstruction K must be stored exactly—using a low-rank-reconstructed key for a residual token was measured to break recall.

4.4. Router statistics

Two cheap summaries per block support decode-time routing (§6): the element-wise key min and max over the block, $k_{\min}, k_{\max} \in \mathbb{R}^{H_{\text{kv}}\times d}$, which bound the best possible query–key score for a Quest-style relevance test.

4.5. Byte budget and complexity

Table 1 accounts for one block exactly as it is laid out in the store. The low-rank core (coefficients, both bases, anchors, min/max, scalars) is a fixed $\approx$ 49.9 KiB, independent of R; the residuals add $R \cdot H_{\text{kv}} \cdot d \cdot 2 \cdot 2$ bytes. At the default R=128 a block is 177.9 KiB versus 256 KiB dense (1.44×); at the R=64 memory preset it is 113.9 KiB (2.25×). The residual budget therefore *is* the compression dial, and the residual pool dominates the block at the default setting—a fact the memory layout makes visual (§5). Compressing a block costs one randomized SVD of an $\approx 255 \times 512$ matrix ($\approx$ 2 ms), amortized over 256 tokens; §8 shows compression is a single-digit percentage of prefill time.

CUDA compression backends. On MLX the randomized SVD runs in NumPy on the CPU with near-zero marshalling cost (unified memory), as noted in §2.5. The CUDA engine exposes two compression paths, selectable at runtime. The *CPU-offloaded* path reuses the identical NumPy SVD, acceptable because compression is a one-time prefill cost per block and the D2H transfer is amortized over B=256 tokens. The *GPU-native* path (`DKV_COMPRESS_GRAM_SVD=1`) replaces the direct SVD with the *Gram-Eigh reformulation*: it computes the $r\times r$ symmetric Gram matrix $BB^\top \in \mathbb{R}^{r\times r}$ and calls `cuSOLVER` `syevjBatched`, which batches correctly at $r \le 32$ (unlike `gesvdjBatched`, which regresses to a sequential per-matrix loop for our $\approx 255\times512$ deltas). Mathematically the right singular vectors of B are the eigenvectors of $BB^\top$, so the factorization is equivalent; controlled CPU timing shows $\approx 6.1\,\text{s} \rightarrow 2.6\,\text{s}$ for a representative prefill. GPU recall validation of this path is pending *(GPU validation pending)*.

An additional fidelity guard applies in both backends: blocks containing high-information-density content (e.g. digit runs matching the pattern `\d{5,}`, detected at compression time) are force-stored with *all* token positions as exact residuals rather than SVD-compressed, bypassing the reconstruction-error selection filter. This preserves verbatim recall of such tokens even when they happen not to top the reconstruction-error ranking.

5. Runtime and Memory Architecture

The store's contents are defined by §4; this section defines its *layout* and *lifecycle*—what makes the runtime's memory bounded and its long-context prefill affordable.

5.1. Session state

Each session holds, per layer (Figure 4): a dense recency window—fixed-capacity fp16 buffers $[H_{\text{kv}},\ W{+}B,\ d]$ for the newest W=1,024 tokens—and a compressed pool

Table 1 | Per-256-token-block storage, computed from the model dimensions and the store's fp16 layout. The low-rank core is fixed; the residual budget R sets the ratio.

Component	Bytes	Note
U coefficients $[255, 32]$	16320	low-rank
V_K, V_V $[2, 32, 128]$	32768	low-rank
anchors a_k, a_v $[2, 128]$	1024	exact
key min/max $[2, 128]$	1024	router
scale, seq_len	8	scalar
Low-rank core	**51144**	(49.9 KiB)
exact residuals R=128	131072	(128.0 KiB)
exact residuals R=64	65536	(64.0 KiB)
DKV block (R=128)	**182216**	**177.9 KiB**
DKV block (R=64)	**116680**	**113.9 KiB**
Dense block $[256, 2, 128]\times2$	262144	256.0 KiB
Compression ratio (R=128)		1.44×
Compression ratio (R=64)		2.25×

Algorithm 1 Compress one block (streaming, batched across layers in the implementation)

Require: block keys/values $K, V \in \mathbb{R}^{H_{\text{kv}} \times B \times d}$; rank r; residual budget R

$a_k, a_v \leftarrow K_0, V_0$ ▷ anchor kept exact

$\Delta K \leftarrow K_{1:} - a_k; \quad \Delta V \leftarrow V_{1:} - a_v$

$g \leftarrow \text{clip}\Big(\sqrt{\|\Delta K\|_F^2 / \|\Delta V\|_F^2},\ 1,\ 10^4\Big)$ ▷ rescale V to K's RMS; clamp guards degenerate blocks

$D \leftarrow \text{rownorm}\big([\,\Delta K \,\|\, g\,\Delta V\,]\big)$; keep norms in U

$U, \Sigma, V^\top \leftarrow \textsc{RandTruncSVD}(D,\ r,\ \text{seed}{=}1234,\ 99.9\%)$

split $V^\top \to V_K, V_V$; undo g on V_V; store scale s

$e_i \leftarrow \|\Delta K_i - \widehat{\Delta K}_i\|^2 + g^2 \|\Delta V_i - \widehat{\Delta V}_i\|^2$ ▷ joint error

apply IDF-weighted boosts: owner-capture, edge-capture, coverage (§4)

$\mathcal{R} \leftarrow$ indices of the R highest-priority e_i; $R_K, R_V \leftarrow K_{\mathcal{R}}, V_{\mathcal{R}}$ ▷ exact

$k_{\min}, k_{\max} \leftarrow \min_i K_i,\ \max_i K_i$ ▷ router bounds

return $(a_k, a_v, U, V_K, V_V, s, R_K, R_V, \mathcal{R}, k_{\min}, k_{\max})$

of M=256 block slots, pre-allocated for U, V_K, V_V, anchors, residuals R_K, R_V, key min/max, and per-block scalars. The window size scales with model capacity: $W = \text{round}_{512}(N_{\text{layers}} \times d \times 0.25)$, which yields 1,024 for Qwen2.5-1.5B (28 layers, d=128). Because the pool is pre-allocated at its maximum size, the store's footprint is *bounded and context-independent* up to the pool's capacity of $256 \times 256 = 65{,}536$ compressed tokens; adding context fills empty slots rather than growing the allocation. This is the structural source of the bounded-memory behavior in §8.

5.2. Chunked, streaming prefill

Prefill consumes the prompt in fixed chunks (default 512 tokens). Each chunk runs causal attention over the native MLX cache to produce correct hidden states, and its K/V are captured into the store. After each chunk, CompressDeferred virtually concatenates the surviving dense tail with the newly captured tokens and compresses every full block that now clears the recency window, in a single SVD batched across all layers and blocks (Algorithm 1 applied in bulk). Streaming compression means peak uncompressed KV during prefill is bounded by roughly one window plus one chunk, not the whole prompt—the difference between fitting a 32k+ prefill in 8 GB and not.

Block-sparse prefill. By default, once a chunk is far enough into the prompt that there is prunable history, its attention is computed over a *sparse* key set rather than the full past: leading attention-sink block(s), a set of top-K routed history blocks (selected by a Quest-style min/max bound on the raw block keys), the recency window, and the current chunk itself, combined under one masked attention call. This is training-free block-sparse attention (attention sinks plus block retrieval) on a frozen model; it reduces prefill attention from $O(L^2)$ toward $O(L \cdot K)$ and is the mechanism behind the prefill crossover in §8 (E3). It changes compute, not the stored state: every token's exact K/V is still captured for later compression, so retrieval accuracy is unaffected.

Prefill memory on discrete GPU (CUDA). On MLX the native prefill KV cache and the DKV block store

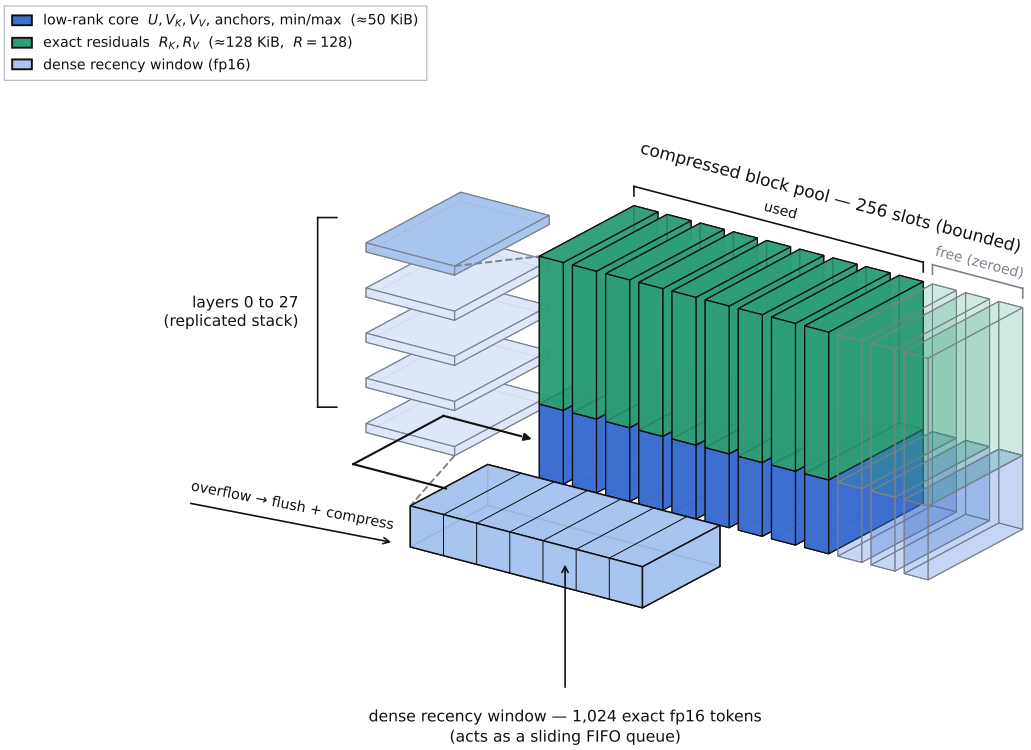

Figure 4 | The per-session KV store in three dimensions. The store is replicated across all 28 layers (left). Within a layer, the compressed pool is a bounded array of M=256 block slots; each occupied slot holds a small low-rank core and—at the default budget—a much larger block of exact residuals (residuals dominate the per-block bytes, cf. Table 1). The dense recency window (front) holds the newest 1,024 tokens uncompressed; when it overflows, its oldest block is flushed and compressed into a pool slot.

coexist in unified memory at minimal extra cost. On a discrete GPU the same dual-copy creates a 2× HBM spike: `past_key_values` (the native cache) plus the DKV store both reside in HBM simultaneously until the prefill→decode boundary. The CUDA runtime offers a *1× contiguous prefill* variant (`DKV_CONTIGUOUS_PREFILL=1`): rather than maintaining a separate native cache alongside the store, it keeps a single RoPE-rotated contiguous buffer and inverse-RoPE at block boundaries when slicing. This eliminates the parallel native-cache copy, halving the peak HBM requirement during prefill. The inverse-RoPE step adds one small matrix multiplication per block boundary and is designed to be a negligible cost at the prefill cadence.

5.3. The prefill→decode boundary

The transition from prefill to decode is where memory is reclaimed (Figure 5). At the first decode step the runtime resolves the decode policy once, flushes pending work with `mx.eval`, and calls `mx.clear_cache` to return the peak GQA-expanded prefill activations to the shared pool. When decode will use the compressed path, it additionally *drops the native prefill KV cache* entirely: decode runs solely on the compressed store plus the dense window, so decode-time memory reflects the DKV footprint and not a retained full-context cache.

5.4. Decode ingestion and the sliding window

Each decode step appends the new token's K/V to the dense window and, if the window has grown past $W+B$, flushes and compresses its oldest block into the pool—the same streaming rule as prefill. The current token is always in the exact window, so a token attends to itself and its immediate neighbors exactly; only older context is served from the compressed pool. The attention itself is the subject of §6.

6. Fused Routed Decode Attention

Decode is where compression must pay off without materializing the cache. The decode kernel (Figure 6) computes, for a single query $q \in \mathbb{R}^{H_q \times d}$, an attention output over the compressed pool and the dense window as two halves merged in log-space. §6.5 then shows how the runtime amortizes that computation across tokens.

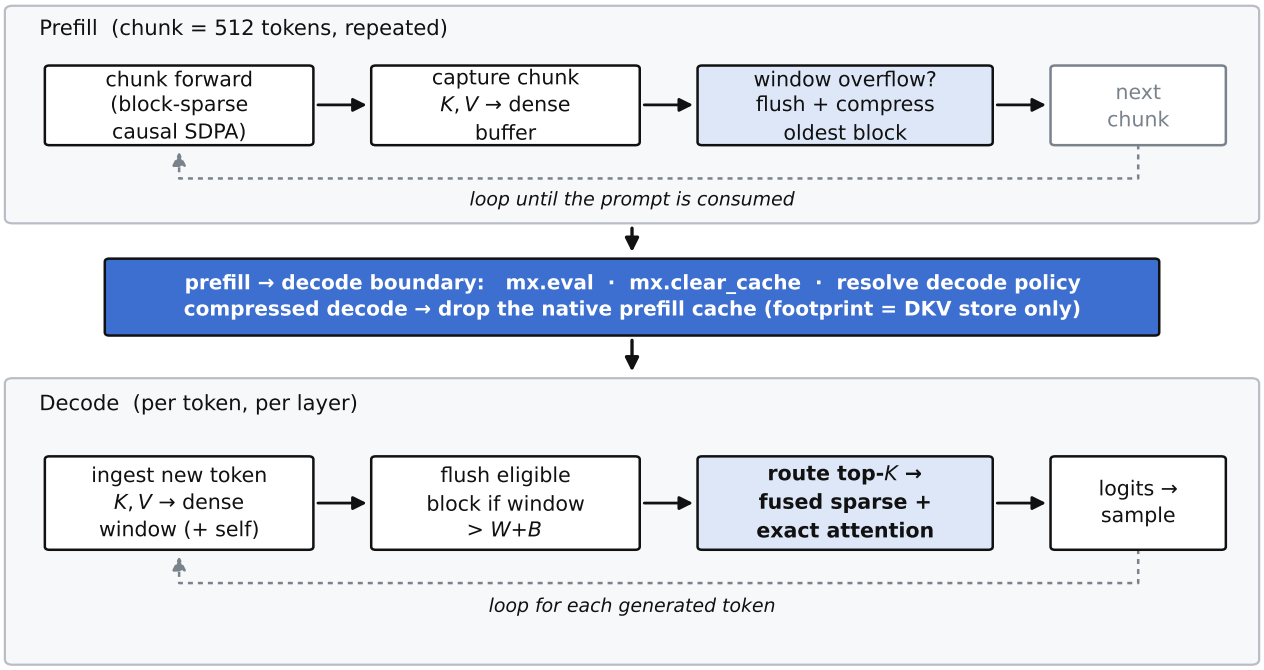

Figure 5 | Cache lifecycle. Prefill loops over chunks (attend, capture, flush-and-compress the oldest block on overflow). At the prefill→decode boundary the runtime evaluates pending work, releases peak activations, and—on the compressed path—drops the native prefill cache. Decode then loops per token: ingest the new token into the window, flush an eligible block, and run the fused routed attention.

6.1. Routing: bound the work, not the context

Attending and reconstructing every block would make decode scale with context. Instead a router scores all blocks cheaply and keeps the top K=16. The default router is *exact*: it scores q against each block's anchor key and its most distinctive residual keys, and ranks blocks by the largest true dot product,

$$\rho_b = \max\Big(q \cdot a_k^{(b)},\ \max_{j \le R} q \cdot R_{K,j}^{(b)} \Big) \cdot \text{scale}. \tag{6}$$

Because the residuals *are* the block's outlier tokens, scoring them gives a tight relevance signal that reliably keeps a needle's block in the top-K even at large block counts—where a min/max box bound (also available, and cheaper, in the style of Quest [14]) becomes too loose. Routing costs $O(\text{nb} \cdot R \cdot d)$ per token and is the dominant decode cost at long context, so the number of residuals *scored* by the router is decoupled from the number *stored* (§7).

6.2. Sparse half: scoring in the low-rank subspace

For each selected block the kernel scores q against every token *without reconstructing* K. Using Eq. (5), the score of token i decomposes into an anchor term and a delta term:

$$q \cdot \hat{K}_i = \underbrace{q \cdot a_k}_{\text{anchor score}} + s \Big(\underbrace{q\, V_K^\top}_{\tilde{q} \in \mathbb{R}^r} \Big) \cdot U_i. \tag{7}$$

The query is projected once into the rank-r subspace ($\tilde{q} = qV_K^\top$), and all $B-1$ token scores in a block are then a single $\tilde{q}\, U^\top$ product. This is $O(K(H_{\text{kv}} r d + rB))$ per token—linear in the rank and independent of d in the inner loop—rather than the $O(K\,B\,d)$ of decompress-then-score. The value side is symmetric: softmax weights are contracted with U and then with V_V, plus the anchor contribution, giving the sparse output O_{sp} and its log-sum-exp lse_{sp}. Residual positions are masked out of this pool so their exact copies (below) are their sole representation.

6.3. Exact half: residuals and recency

The selected blocks' exact residual keys/values are concatenated with the dense recency window and attended directly in fp16—an ordinary masked attention producing O_{dn} and lse_{dn}. This half is where a buried passcode is recovered verbatim: its token, being a high-error outlier, was kept exact as a residual, so the query attends to the true key and copies the true value.

Algorithm 2 Session lifecycle (one request)

init session: pre-allocate dense window + pool of M block slots (per layer)
for each prefill chunk c **do** ▷ L>1
  attend(c) over native cache (block-sparse if enough history); capture K, V
  CompressDeferred: compress full blocks clearing the window (batched SVD)
end for
boundary: resolve decode policy; `mx.eval`; `mx.clear_cache`
if compressed decode **then** drop native prefill KV cache
end if
for each decode step **do** ▷ L=1
  ingest new token K, V into dense window; flush+compress oldest block if overflow
  $o \leftarrow$ FusedRoutedAttention(store, q) ▷ Alg. 3
  sample next token from logits
end for

6.4. Flash-style merge

The two halves are combined by the standard online-softmax identity used by FlashAttention [3, 2], in a numerically guarded form:

$$m = \max(\mathrm{lse}_{\mathrm{sp}}, \mathrm{lse}_{\mathrm{dn}}), \quad o = \frac{e^{\mathrm{lse}_{\mathrm{sp}} - m} O_{\mathrm{sp}} + e^{\mathrm{lse}_{\mathrm{dn}} - m} O_{\mathrm{dn}}}{e^{\mathrm{lse}_{\mathrm{sp}} - m} + e^{\mathrm{lse}_{\mathrm{dn}} - m}}. \tag{8}$$

NaN/inf guards make an empty compressed set contribute exactly zero weight, so the kernel degrades to pure dense-window attention with no special case. We keep the score and combine arithmetic in the precision the kernel was validated at (fp16 products with fp32 log-sum-exp accumulation); recasting the merge to fp32 was measured to perturb long-context retrieval and is avoided (§7). An optional additive bias on the compressed half's log-sum-exp corrects a systematic underweighting of the low-rank pool for diffuse multi-fact queries; it defaults to a no-op and does not affect the exact single-needle recall reported here.

6.5. Fused decode buffer

Reconstructing the routed blocks for every token is the dominant per-token cost. To amortize it, the runtime uses a *persistent fused buffer* (Figure 7): at each routing interval (default N=16 tokens), the selected blocks' K/V are materialised into a contiguous static buffer —one row per non-masked token—together with their exact residuals and the dense recency window, guarded by a fixed additive mask that gives residual positions infinite score and zeroes out the low-rank twins of residual rows so each token is represented once. Per-token decode then appends the new token's K/V as a single in-place row and runs one `scaled_dot_product_attention` call over exact-length views, without any per-token concatenation or buffer re-allocation. The block set is re-routed and the buffer re-written every N tokens (or when the set changes). This path (`DKV_DECODE_FUSED`, default on) is bit-exact to the per-token sparse kernel; its effect on throughput is part of the measured decode numbers in §8.

6.6. Backend dispatch and kernel implementations

The decode algorithm (Algorithm 3) is implemented by three independent kernel paths that share the identical compressed block format (Figure 8).

MLX/Metal path (evaluated). The MLX implementation uses `mx.matmul` for the query projection and U-expand, `mx.fast.metal_kernel` for tiled UV expansion across Metal threadgroups on Apple Silicon (with a fallback to `mx.matmul` if the MLX version does not expose the custom-kernel API), and `scaled_dot_product_attention` for the dense-window and residual half. This is the path benchmarked throughout §8.

CUDA/Triton path *(GPU validation pending)*. `native_triton_sparse_attn_decode()` dispatches to `_fused_sparse_decode_kernel`, a Triton kernel that fuses the anchor score, delta projection ($q\,V_K^\top \rightarrow \tilde{q}$, then $\tilde{q} \cdot U_i$ for all i), and exact residual correction in one kernel launch (Figure 10). Residual corrections are applied *in place*: the kernel scatters $q \cdot R_K$ directly into the accumulated score at position `res_pos`, and scatters $p \cdot R_V$ into the output numerator at `res_pos_v`, *before* the block's softmax denominator is finalized. This is architecturally distinct from *appending* residuals as extra softmax tokens: appending inflates the denominator with a ghost entry; in-place correction does not (Figure 9), and the reference decoder confirms the difference is numerically significant. K and V use *separate* position arrays (`res_K_positions` and `res_V_positions`), because the highest-error rows differ between the two projections. The kernel is autotuned via `@triton.autotune` over `(S_MAX,BLOCKS_PER_CHUNK,num_warps)`: on the first five real-shape calls the configurations are benchmarked and the fastest is locked in, giv-

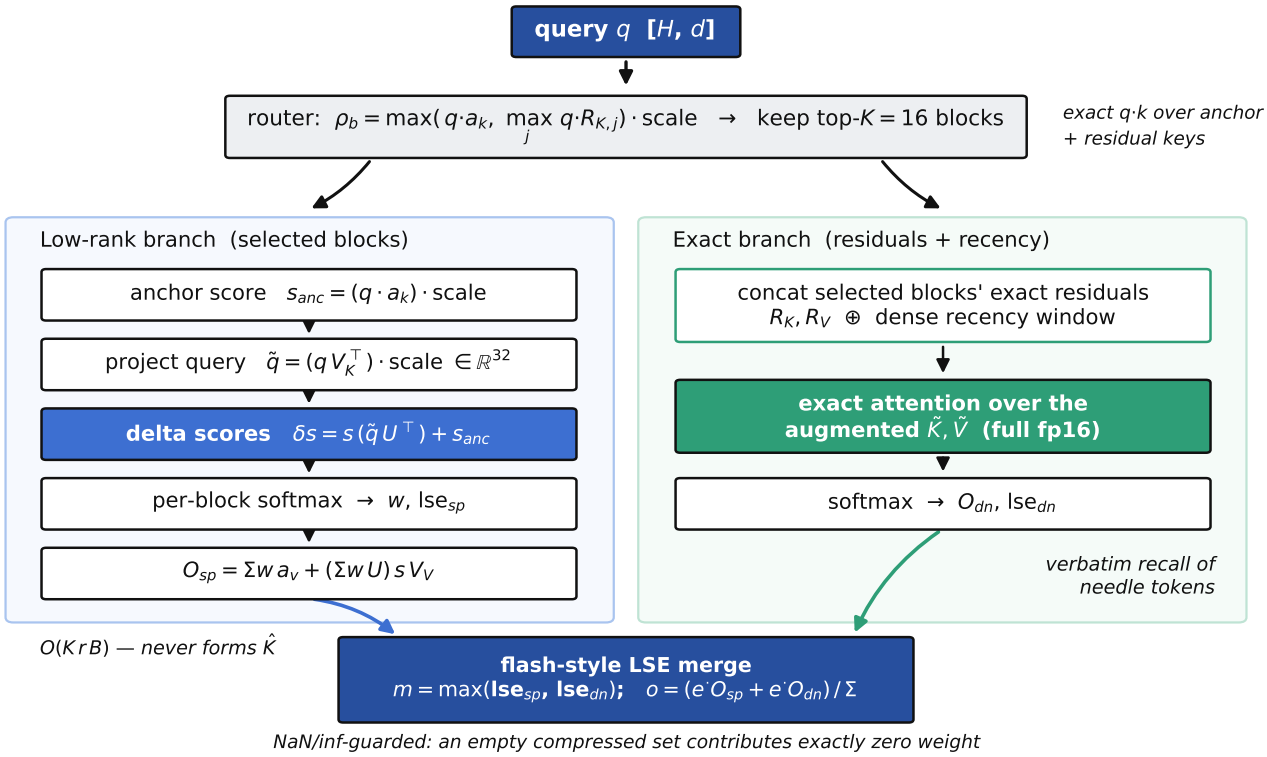

Figure 6 | The routed sparse decode kernel. A cheap exact router keeps the top-K blocks. Their contribution is computed in the rank-r subspace—anchor score plus a projected-query dot with U—without ever forming $\hat{K}$. In parallel, the selected blocks' exact residuals and the dense recency window are attended directly. The two halves are combined by a flash-style log-sum-exp merge, guarded so an empty compressed set contributes exactly zero weight.

ing full inner-loop unrolling matched to the specific (r, d) shape. A PyTorch vectorized fallback (`_pytorch_vectorized_sparse_attn_decode`) is available for testing or if Triton compilation fails; `DKV_TRITON_STRICT=1` converts the fallback to a hard error, which is recommended for GPU regression testing so a wrong-but-non-throwing kernel is not silently measured.

Native C++/GGML path *(GPU validation pending)*. `execute_cuda_attention()` calls `dkv_full_decode_kernel` in `dkv_decode.cu`, compiled with `-DGGML_CUDA=ON`. It implements the same anchor score, per-token delta projection, and K/V separate-position in-place residual correction as the Triton path, in pure CUDA C++. The split-K scratch tensors are sized dynamically from `n_q_heads` × `S_split` × `D` at runtime and reallocated on model change. A `DKV_CUDA_CHECK` macro wraps every `cudaMalloc`, `cudaMemcpy`, and kernel launch, with `cudaGetLastError()` after each launch; `DKV_CUDA_ANCHOR_ONLY=1` reverts to the anchor-only stub for A/B regression testing.

7. Implementation Notes

DKV's active runtime is ≈ 6,000 lines of Python over MLX [7], evaluated on Qwen2.5-1.5B [13]. A few implementation choices are load-bearing and worth recording, because they are where correctness or performance was actually won or lost.

Attention by monkey-patch. Rather than fork `mlx_lm`, the runtime replaces `qwen2.Attention.__call__` at load time with the patched forward of §3, and hands each layer a reference to the shared block manager. This keeps the runtime a thin, model-agnostic overlay on an unmodified upstream model.

Randomized SVD in NumPy. Compression runs the randomized truncated SVD in fp32 in NumPy on the CPU, not in MLX on the GPU. On unified memory this is nearly free to marshal (no host–device copy), and it sidesteps GPU-only limitations in the current MLX linear-algebra path (e.g. `qr`); a 255×512 block factorizes in ≈ 2 ms. The seed is fixed so that compression—and therefore decode output and needle recovery—is deterministic run to run; an unseeded range finder made recall vary between identical runs.

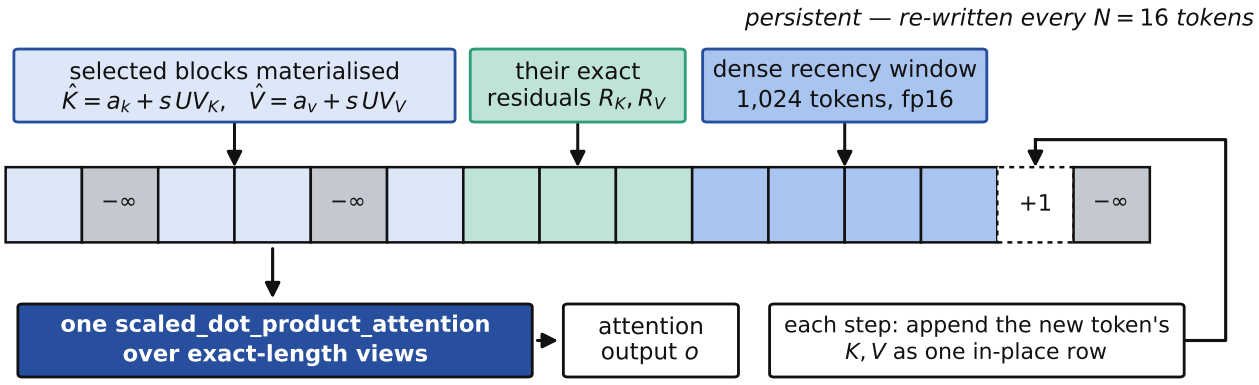

Figure 7 | The fused decode buffer. Every N=16 tokens the routed blocks' reconstructed rows, their exact residuals, and the dense recency window are written into one pre-allocated contiguous buffer. Grey cells score $-\infty$ under the fixed additive mask—the low-rank twins of the residual rows, and the unused tail—so each token is represented exactly once. Between rewrites, a step appends the new token's K/V as a single in-place row and issues one `scaled_dot_product_attention` over exact-length views. Attending the union of these rows in one softmax is algebraically the log-sum-exp merge of the two halves in Figure 6.

Algorithm 3 Fused routed decode attention for query q (one layer)

Require: query q; pool $\{U, V_K, V_V, a_k, a_v, s, R_K, R_V\}_b$; dense window K_d, V_d

$\rho_b \leftarrow \max(q \cdot a_k^{(b)}, \max_j q \cdot R_{K,j}^{(b)}) \cdot \text{scale}$ ▷ Eq. (6)

$\mathcal{S} \leftarrow$ top-K blocks by ρ_b

for $b \in \mathcal{S}$ **do** ▷ scored in rank space, no $\hat{K}$

  $\tilde{q} \leftarrow q\, V_K^{(b)\top}$; $\text{scores}_i \leftarrow q \cdot a_k^{(b)} + s\, \tilde{q} \cdot U_i^{(b)}$

  mask residual positions to $-\infty$

end for

$O_{\text{sp}}, \text{lse}_{\text{sp}} \leftarrow$ softmax/combine over pooled block scores with V side of Eq. (5)

$O_{\text{dn}}, \text{lse}_{\text{dn}} \leftarrow$ exact attention over $[\, R_{K,\mathcal{S}} \,\|\, K_d \,], [\, R_{V,\mathcal{S}} \,\|\, V_d \,]$

return flash-merge$(O_{\text{sp}}, \text{lse}_{\text{sp}}, O_{\text{dn}}, \text{lse}_{\text{dn}})$ ▷ Eq. (8)

Decode precision. The decode score/merge arithmetic uses fp16 products with fp32 log-sum-exp accumulation. A well-intentioned change that recast the merge operands to fp32 for "overflow safety" was measured to shift the combine by roughly fp16-epsilon and *regress* long-context (16k/32k) needle recall into a repetition loop, because at those lengths the retrieval margin sits on a knife edge. The fp32-log-sum-exp / fp16-combine split is the validated configuration and is enforced by a parity oracle in the test suite.

Router/attention decoupling. The number of residuals the router *scores* (Eq. (6)) is a separate knob from the number a block *stores*. The router cost is $O(\text{nb} \cdot R_{\text{route}} \cdot d)$ per token and dominates long-context decode, so R_{route} defaults to $\min(64, R)$ rather than the full storage budget; raising R for prose fidelity therefore does not automatically inflate router cost.

Layer-adaptive rank. By default, the SVD rank assigned to each layer varies with depth: early layers ($\leq 25\%$) use $0.75r$, middle layers (25–75%) use $1.5r$, and late layers ($\geq 75\%$) use $0.5r$, where r=32 is the base rank. The pool is sized at the maximum rank $1.5r$=48; layers use only what they need. Attention quality is more rank-sensitive in middle layers (higher information density), while early and late layers tolerate more aggressive compression. The base rank r=32 is the value cited throughout.

Multi-signal residual selection. Residual tokens are selected by a boosted-priority ranking rather than plain reconstruction error. Three IDF-weighted boost signals are active by default: *owner-capture* (entity names accompanying high-error values), *edge-capture* (relational connectives whose low-rank reconstruction collides with a neighbour's key, detected by a cosine-collision test), and a *coverage bonus* (uniform spread across block positions so a single high-error segment cannot monopolise all slots). All three are individually controllable via environment flags; together they preserve the entity-name and relational structure that the error signal alone cannot reliably surface.

Configuration. Behavior is controlled by environment flags with safe defaults, so new paths ship disabled

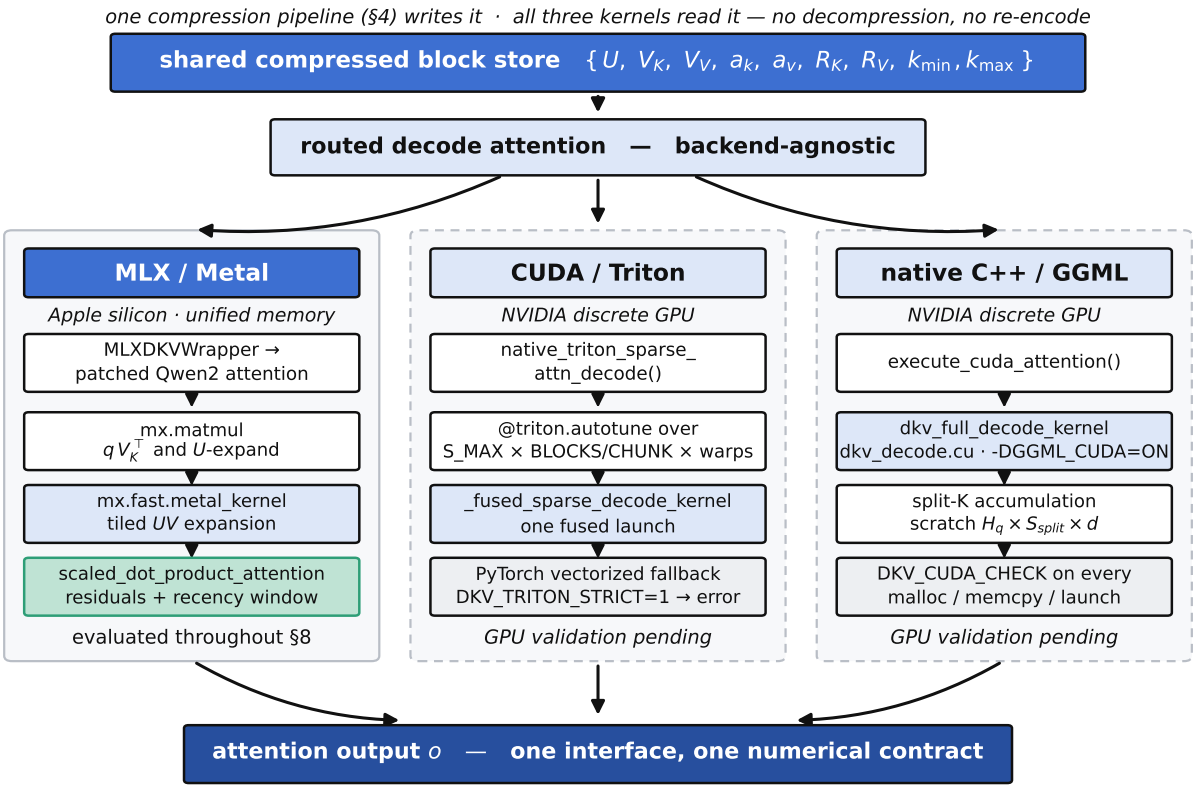

Figure 8 | Backend dispatch. One compression pipeline (§4) writes the block store; three independent kernels read it and return the same attention output through the same interface. They are three implementations of Algorithm 3, not three algorithms: no path decompresses, re-encodes, or otherwise reinterprets the stored format. Figure 2 details the MLX column, which is the path evaluated in §8; the two CUDA columns are implemented but GPU validation is pending *(GPU validation pending)*.

until validated. The defaults used throughout this report are: compressed sparse decode on; fused decode buffer on (`DKV_DECODE_FUSED`); block-sparse prefill on; base rank r=32 with layer-adaptive variation; R=128; top-K = 16; residual-key router; SVD seed 1234. The optimized dense baseline is `mlx_lm` with a full KV cache (chunked prefill, prefill-to-decode allocator release) on the same int4 weights; the standard PyTorch baseline is `AutoModelForCausalLM` on the PyTorch MPS backend with an uncompressed full KV cache and none of those memory-management paths, loading the unquantized fp16 checkpoint (`Qwen/Qwen2.5-1.5B-Instruct`) because `transformers` cannot read the MLX int4 format—so it is a practitioner-default reference point, not a weight-matched control (§8, §10).

Gated subsystems. A factual-store / semantic-retrieval module, a lego streaming prefill (memory-bounded prefill that drops raw KV after block compression), a speculative-decode draft path, and a query-aware instruction-pinning mechanism (`DKV_INSTR_PIN`) exist in the codebase but are disabled by default (they are no-ops in the configuration measured here) and are not part of the evaluated system. Query-aware instruction pinning identifies high-IDF query tokens pre-prefill to statically pin answer-candidate blocks as always-attended attention sinks during block-sparse prefill, avoiding question-block pruning when instructions appear mid-document; we implement this as an architectural extension for future long-context instruction-following ablations. We mention these subsystems only to be exact about what is and is not exercised.

Runtime optimizations inspired by kTransformers. The active runtime incorporates four throughput and memory optimizations whose design is informed by the kTransformers inference framework [18], adapted to DKV's compressed-block layout.

(i) Tiered CPU–GPU KV offloading. kTransformers demonstrates that heterogeneous CPU–GPU memory tiers can sustain long-context inference by keeping hot layers in GPU memory while cold weights reside in CPU RAM [18]. We apply the analogous idea at the *block* level: each pool slot carries a heat score $h \leftarrow 0.7\,h\,\gamma + 0.3\,\rho$ (decay γ=0.95, routing signal $\rho \in [0, 1]$) updated at every decode step. When pool occupancy exceeds a threshold (default 80%), the coldest slots are evicted to pinned CPU RAM and their GPU memory is reclaimed. Because

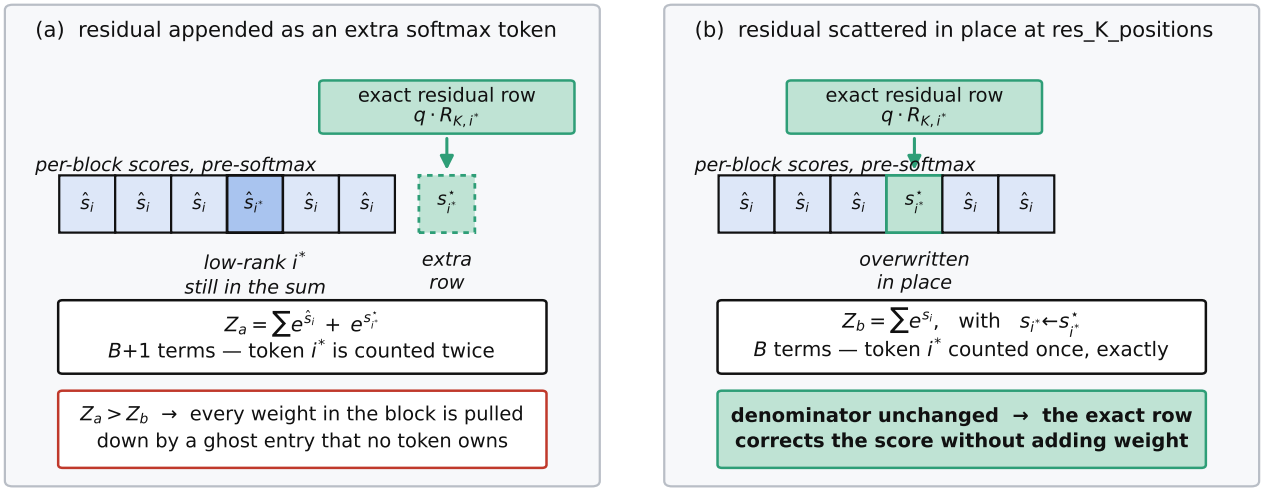

Figure 9 | Why the residual correction is applied *in place*. *(a)* Appending the exact row as an extra softmax token leaves the token's low-rank approximation in the sum as well, so i^* contributes twice and the denominator carries a B+1-th term that belongs to no token; every weight in the block is scaled down by it. *(b)* Scattering $q \cdot R_K$ into the score already held at position i^* replaces the approximate value before the denominator is finalized, leaving B terms. K and V use separate position arrays (`res_K_positions`, `res_V_positions`) because the highest-error rows differ between the two projections.

DKV's residual-key router (Eq. (6)) selects the same block neighborhood across consecutive decode steps, the same K blocks remain hot and almost never enter the cold tier during steady-state generation; offloading triggers only for blocks that have not been routed for many steps, i.e. genuinely irrelevant history. Re-routing to a previously cold block requires a host-to-device (H2D) transfer; for a default block (U: 63×32 int8; V_K, V_V: 2×32×H_{kv}×d fp16; anchors: fp16) the total is ≈137 KB, costing ≈15 μs on PCIe 4.0. At the decode cadence of 20–80 ms per token this is below 1% overhead even for a cold hit, and Feature (ii) below hides it entirely.

(ii) Step-ahead asynchronous block prefetch. After the router delivers the top-K slot indices for step t, a background daemon thread submits non-blocking H2D restores for any cold slots in that set, so they are warm before step t+1 begins. The submit path is a lock-free `queue.put_nowait` call (≈50 ns) and does not touch the critical path. On CUDA, the restore issues on a dedicated `stream_h2d` stream; after issuing the `cudaMemcpyAsync` the dispatcher records a `cudaEvent` on `stream_h2d` and calls `cudaStreamWaitEvent` on the main compute stream, so the kernel launch on step t+1 automatically stalls only as long as the H2D transfer is outstanding—no global device synchronization. On MLX/MPS the transfer is synchronous but still below the decode step budget. In practice, because routing is stable across steps, the prefetch almost always completes before it is needed, making the effective H2D overhead zero.

(iii) Kernel-level tiling for the expansion step. On CUDA, the main fused decode kernel (§6) is wrapped with `@triton.autotune` [15], which benchmarks a grid of (`S_MAX`, `BLOCKS_PER_CHUNK`, `num_warps`) configurations on the first five calls and thereafter uses the fastest for each (r, d) shape. This gives full inner-loop unrolling and register reuse matched to the specific rank and GPU. On Apple Silicon the analogous acceleration uses `mx.fast.metal_kernel` to tile the UV expansion across Metal threadgroups, matching the throughput profile of a hand-written tiled matmul; it falls back to standard `mx.matmul` if the MLX version does not expose the custom-kernel API.

(iv) MLA-style latent projection (experimental, disabled by default). DeepSeek-V2 [4] introduces Multi-head Latent Attention (MLA), which projects keys and values into a shared low-rank latent space before storage, reducing KV cache footprint without architecture changes [4]. We incorporate an analogous pre-SVD projection: a learned matrix $W \in \mathbb{R}^{d_{\text{feat}} \times d_L}$ (with $d_L \ll d_{\text{feat}}$, default $d_L = d_{\text{feat}}/4$) projects the per-block delta matrix to a latent space before the SVD, exposing a more concentrated singular spectrum and reducing the effective rank needed to reach the energy threshold. W is initialised from an online PCA over the first n_{calib} blocks (default 16), after which it is frozen. The right singular vectors are unprojected back to full d_{feat} before pool storage so the attention kernel interface is unchanged. Before W converges the projection is the identity, preserving correctness during warm-up. Because the additional reconstruction error (≈0.1–0.5%) compounds with the SVD approximation, this feature is disabled by default (`DKV_MLA_LATENT=0`) and requires NIAH recall validation before production use.

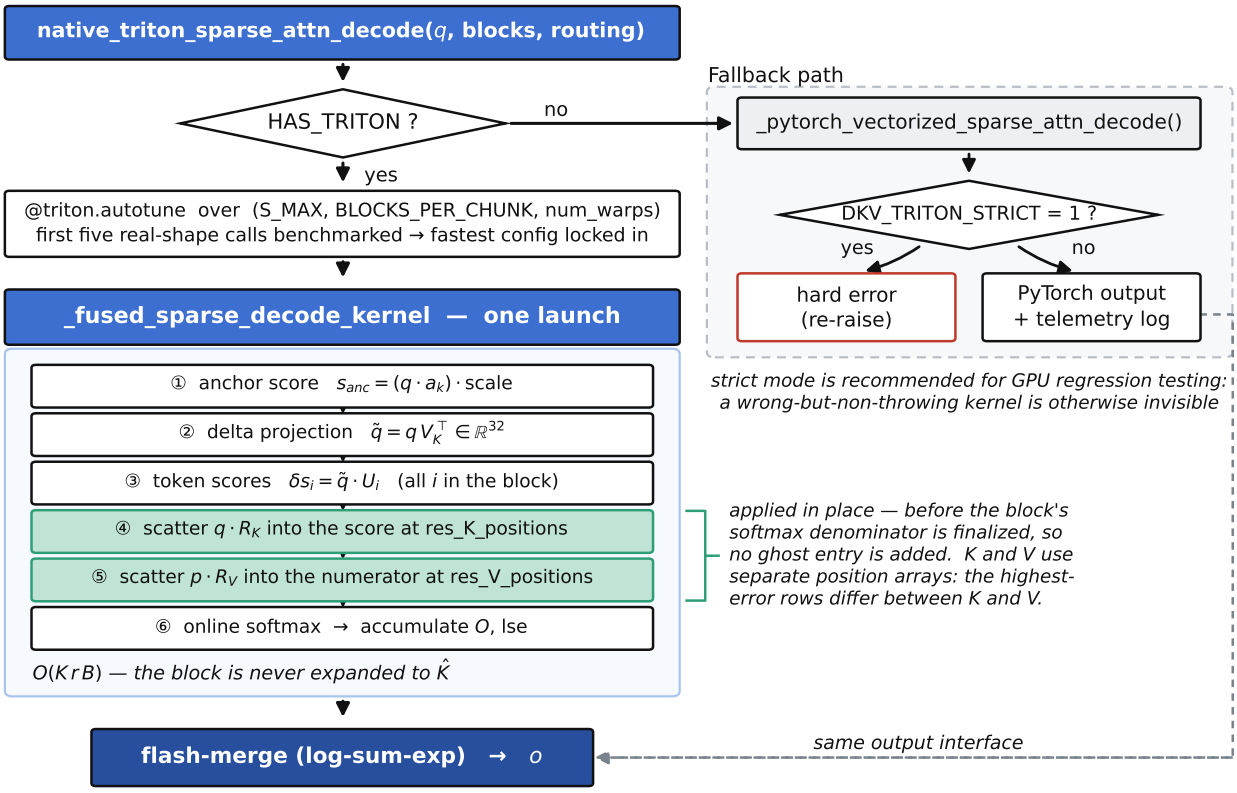

Figure 10 | CUDA/Triton decode dispatch. When Triton is available the six kernel steps run in a single autotuned launch, with the exact residual rows scattered in place at steps 4–5 (Figure 9). When it is not, a PyTorch vectorized fallback returns through the same interface—which is why `DKV_TRITON_STRICT=1` exists: it converts the fallback into a hard error so a GPU regression run cannot silently measure the wrong path. *(GPU validation pending)*

Testing. A kernel-parity oracle checks the fused decode kernel against a reference implementation on seeded sessions; the needle and relational-binding harnesses guard recall. The measurement scripts, prompts, and the exact commands are in the reproducibility appendix.

7.1. CUDA-specific implementation notes

The kTransformers-informed optimizations above (§7) span both backends. This subsection records three CUDA-only design decisions that arise from the discrete-GPU memory model and have no MLX equivalent.

Triton compilation pipeline and strict-mode telemetry. At import time, `triton_fused_decode.py` checks `HAS_TRITON`; if true, the kernel is compiled with `@triton.jit` on the first call. A one-time log line—**"Triton path ACTIVE"**—confirms the live kernel is executing rather than the PyTorch fallback. This distinction is critical for GPU regression testing: a numerically wrong kernel that does not throw an exception is otherwise invisible, and a performance measurement taken on the PyTorch path would silently undercount CUDA throughput. `DKV_TRITON_STRICT=1` converts the `exceptException` fallback into a hard re-raise, making such failures explicit. The default (`STRICT=0`) preserves safe degradation in production environments where Triton is absent.

GPU-native SVD via the Gram-Eigh reformulation. As described in §2.5 and §4, CUDA compression can run the same CPU-offloaded NumPy SVD as MLX, or a GPU-native path activated by `DKV_COMPRESS_GRAM_SVD=1`. The reformulation computes the $r \times r$ Gram matrix $BB^\top$ and calls `cuSOLVER syevjBatched`. Two additional flags tune the trade-off: `DKV_RSVD_MAX_RPROJ=32` caps the Gram-matrix dimension at 32 for `syevjBatched` compatibility (larger ranks fall back to CPU), and `DKV_CONTIGUOUS_PREFILL=1` / `DKV_CONTIG_UNROTATE=1` activate the 1× prefill variant described in §5 that eliminates the 2× HBM spike. GPU recall validation of the Gram-Eigh path is pending *(GPU validation pending)*.

CUDA Graphs: roadmap and current blockers. CUDA Graphs would allow the per-token decode sequence to be captured as a single replayed graph, eliminating per-kernel launch overhead. The current implementation cannot use them because host Python synchronization points—`tensor.item()` and `routing.tolist()` calls in

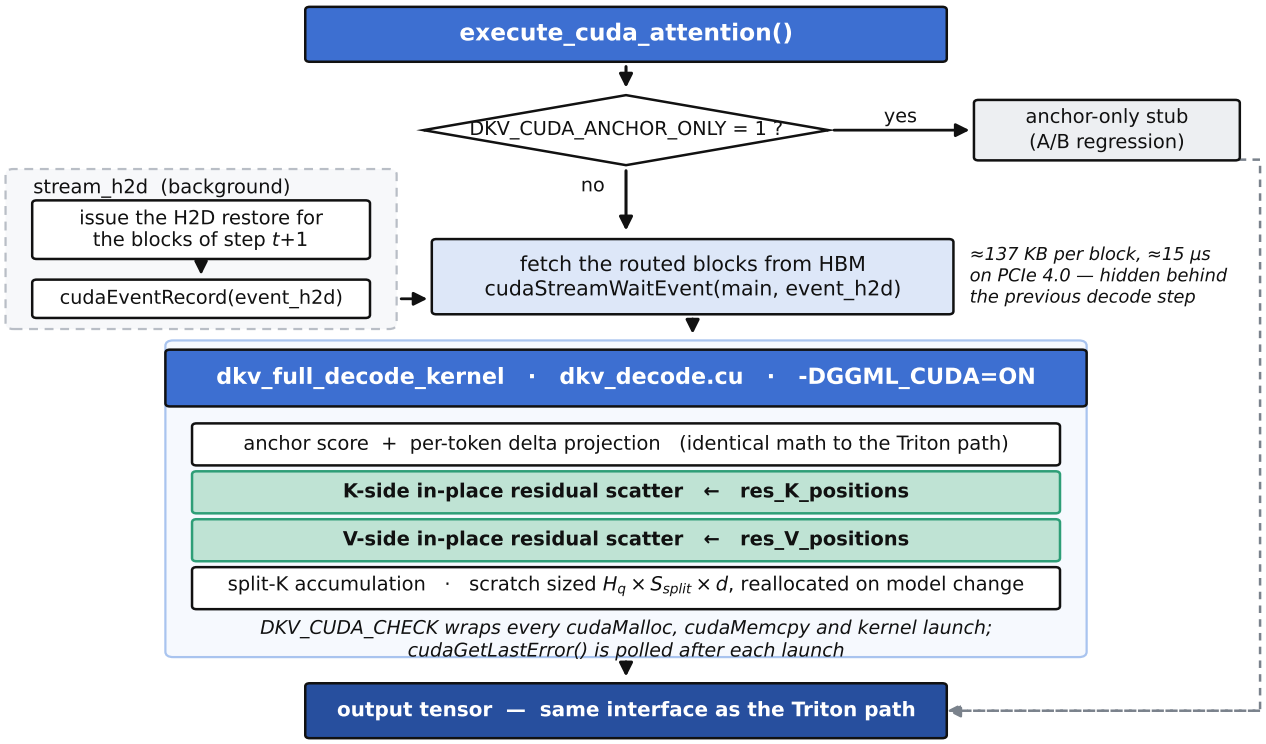

Figure 11 | Native C++/GGML decode path. `execute_cuda_attention()` waits on the event recorded by the background `stream_h2d` prefetch, so the H2D block restore (§2.5, Figure 1) is already in flight a step ahead. The kernel body performs the same anchor score, delta projection and separate K/V in-place residual scatter as the Triton path (Figure 10) and returns through the same interface—the two are one algorithm in two languages, differing only in the split-K scratch allocation and the `DKV_CUDA_CHECK` wrapper. *(GPU validation pending)*

the routing hot path—occur mid-graph and require host–device round trips that prevent graph capture. The planned migration is a three-phase sequence: (1) replace all `.item()` / `.tolist()` calls in the critical decode path with async-safe GPU predicates so the routing decision stays on device; (2) isolate the graph boundary to the pure-kernel sequence (query projection, Triton fused kernel, inter-chunk reduction kernel); (3) validate graph replay bit-exactness against eager mode on seeded sessions before enabling for production. No step in this roadmap requires changes to the block representation or compression algorithm.

8. Evaluation

8.1. Setup

All numbers are measured on one host: an Apple M3 with 8.6 GB of unified memory, running Qwen2.5-1.5B. We compare three engines: **DKV** (the active runtime in its default serving configuration—compressed sparse decode, fused decode buffer, block-sparse prefill, base rank r=32 with layer-adaptive variation, R=128, K=16); **optimized dense** (`mlx_lm` with a full KV cache, chunked prefill, and a prefill-to-decode allocator release); and **standard PyTorch dense** (`AutoModelForCausalLM` on the PyTorch MPS backend with an uncompressed full KV cache and none of the chunking or allocator tricks—the naive baseline most practitioners run).

DKV and the optimized dense engine run on *identical* int4 weights (`mlx-community/Qwen2.5-1.5B-Instruct-4bit`), so that pair is a clean ablation of the cache representation and its memory management, isolated from the model; the optimized dense engine is the demanding baseline, and it reaches every context we test. The standard PyTorch engine is a different kind of reference point and we are explicit about what it is not: it loads the *unquantized* fp16 checkpoint (`Qwen/Qwen2.5-1.5B-Instruct`, `torch_dtype=float16`), because the MLX int4 checkpoint is not loadable by `transformers`. Its weights therefore cost ≈ 3.1 GB rather than ≈ 1 GB, and that difference—not the KV cache alone—is part of why it exhausts this host earlier. We report it because it is the configuration a practitioner gets by default on this hardware, not as a controlled ablation; *the controlled comparison is DKV versus the optimized dense engine*, and no claim in this paper rests on the PyTorch engine alone.

The workload is a needle-in-a-haystack (NIAH)

prompt [8]: a unique passcode is buried at ∼50% depth in a technical-prose haystack, and the request asks the model to state the passcode and then write a long explanation. The same deterministic prompt is used for every context length and all three engines. We report prefill time, decode throughput (greedy, a fixed 128 tokens with EOS ignored so throughput is comparable), peak memory, and whether the exact passcode appears in the output. Contexts span 4k to 64k. For the MLX engines, memory is the MLX allocator peak (`mx.get_peak_memory`), the allocator-true metric; for the PyTorch engine it is `torch.mps.driver_allocated_memory()`, the MPS driver's allocation high-water mark, which is the closest available analogue but is not the same metric—the two columns should not be compared to two decimal places. The analytic KV-state footprint is computed from the store layout (Table 1); for the PyTorch engine it is likewise analytic (b_{tok} from Eq. (1) times the token count) rather than read from an allocator. Table 2 is the headline.

A caveat on the PyTorch prefill/decode split. PyTorch's MPS backend executes asynchronously, and the harness does not insert a `torch.mps.synchronize()` before it stops the prefill timer—the first host synchronisation is the `.item()` call inside the decode loop. Prefill work is therefore charged to the decode phase, which is visible in the data: the measured PyTorch prefill *falls* from 0.83 s at 4k to 0.50 s at 8k, which cannot be a real prefill curve. We report the numbers as recorded rather than silently adjusting them, but the PyTorch engine's prefill and decode figures should be read only as an end-to-end total (≈ 36 s at 4k for prompt plus 128 tokens, against ≈ 6.9 s for the optimized dense engine); its *split* between the two phases is not meaningful, and no claim in this paper depends on it. The MLX engines are timed around `mx.eval` and do not have this problem.

Evaluation scope. All numbers in this section are measured on MLX on the Apple M3 host described above. The CUDA/Triton and native C++/GGML backends exist and are described in §6.6 and §7.1, but no GPU benchmark has been collected; no CUDA throughput, memory, or recall numbers appear in this section. The result tables follow a column convention that accommodates a future CUDA/GPU baseline without restructuring.

8.2. E1 — KV-state footprint

Figure 12 plots the analytic KV state against context. A dense cache grows linearly and without bound; DKV's store grows more slowly and is capped by the fixed 256-block pool. At the default R=128 the per-block ratio is 1.44×; the R=64 memory preset reaches 2.25× and keeps the whole store under the pool cap even at 64k. The point is not a large constant factor—the residual pool deliberately spends bytes to preserve recall—but that the state is *bounded*: past the pool's capacity the cache stops being the term that grows, which is what decides whether long context fits on a memory-constrained device.

8.3. E2 — Exact recall and reach

Across every context from 4k to 64k, DKV recovers the buried passcode *exactly* (Table 2, "Needle recovered"); both dense baselines also recover it wherever they run. This is the load-bearing correctness result for DKV: compression to a low-rank basis would ordinarily corrupt such a token, and the residual mechanism (§4) is what preserves it. The *reach*, however, differs sharply by engine. The standard PyTorch full-KV baseline *exhausts the host's 8.6 GB and does not complete beyond 8k*—a reproducible out-of-memory at 16k, where its fp16 weights, its uncompressed cache and its unchunked prefill together overrun the machine. We attribute that failure to the combination, not to the cache alone: at 16k the cache is 0.47 GB while the fp16 weights already cost ≈2 GB more than the int4 weights the other two engines use. The optimized `mlx_lm` dense cache, with chunked prefill and a prefill-to-decode allocator release, stretches all the way to 64k; DKV reaches 64k as well, finishing with the passcode intact at a 5.04 GB allocator peak. So on this host the default PyTorch configuration gives out at 16k—a 4× shorter reach—while DKV's bounded store runs to 64k, the same context a carefully memory-managed dense can also hold but which it prefills markedly more slowly (E3). The load-bearing comparison for the cache representation itself is DKV against the optimized dense engine on identical weights, where the difference is a bounded versus an unbounded store rather than a reach cliff.

8.4. E3 — Prefill scaling and the crossover

DKV's prefill does less attention work than dense (block-sparse, $O(L \cdot K)$ vs. $O(L^2)$), so its cost grows sub-quadratically while dense grows quadratically (Figure 13a). At short context the fixed cost of streaming SVD compression makes DKV slightly slower (6.2 s vs. 5.0 s at 4k); the two are essentially matched by 16k (29.8 s vs. 27.0 s), and DKV pulls ahead by 32k, prefilling in 72.3 s versus dense's 75.0 s (1.04×). The crossover is where the quadratic term in dense attention overtakes the fixed compression overhead, and it moves earlier as either the model or the context grows. At 64k the gap widens into DKV's clearest prefill win: it completes prefill in 477 s against the optimized dense baseline's 821 s—a 1.72× speedup—both engines heavily memory-bound on this 8.6 GB host, so the absolute 64k seconds are reach points more than clean timings. The standard PyTorch dense never enters this regime at all: it OOMs at 16k (E2), so the only engines still prefilling at 64k are DKV and the memory-optimized dense, with DKV the faster of the two.

Table 2 | Main results: DKV vs. an optimized `mlx_lm` dense baseline (both on identical int4 weights) and a standard PyTorch dense baseline (unquantized fp16 weights; see §8 setup). Apple M3, 8.6 GB; greedy, 128 decode tokens. Timing, throughput and peak-memory cells are measured; KV-cache footprint is analytic from the store layout (Table 1) and Eq. (1). The PyTorch prefill/decode *split* is distorted by an unsynchronised timer (§8); read that row's two timing cells only as an end-to-end total.

Metric	4k	8k	16k	32k	64k
DKV active runtime (compressed sparse decode)					
Prefill (s)	6.2	13.4	29.8	72.3	477.1
Decode (tok/s)	33.3	31.4	29.7	27.2	21.4
KV cache footprint (GB)	0.07	0.15	0.28	0.56	1.12
Peak process memory (GB)	1.55	1.67	2.20	3.12	5.04
Needle recovered	✓	✓	✓	✓	✓
Optimized dense baseline (mlx_lm, full KV)					
Prefill (s)	5.0	11.1	27.0	75.0	821.0
Decode (tok/s)	68.9	54.5	48.2	37.5	20.2
KV cache footprint (GB)	0.12	0.24	0.47	0.94	1.88
Peak process memory (GB)	1.68	1.79	2.03	2.45	3.23
Needle recovered	✓	✓	✓	✓	✓
Standard PyTorch dense baseline (AutoModelForCausalLM, full KV)					
Prefill (s)	0.8	0.5	*OOM*	*OOM*	*OOM*
Decode (tok/s)	3.6	3.5	*OOM*	*OOM*	*OOM*
KV cache footprint (GB)	0.12	0.24	*OOM*	*OOM*	*OOM*
Peak process memory (GB)	8.05	8.05	*OOM*	*OOM*	*OOM*
Needle recovered	✓	✓	*OOM*	*OOM*	*OOM*

8.5. E4 — Decode throughput (the cost)

We state the cost of compression plainly. DKV's decode reconstructs and merges a sparse representation every token, which a dense cache does not; where the optimized dense cache still fits in memory—which, for this 1.5B model on 8.6 GB, holds through 64k—dense decodes faster at the lengths it is comfortable at (Figure 13b, Table 2). With the fused decode buffer enabled, DKV decode is nearly *flat* across context (33.3 → 27.2 tok/s from 4k to 32k—the top-K routing bounds the per-token work independent of length and the buffer is re-materialised once per interval), while the optimized dense declines steadily (68.9 → 37.5 tok/s as its cache grows). The gap therefore *narrows* as context grows—from over 3× at 4k to 1.4× at 32k, and by 64k DKV's flat curve and the still-declining dense curve essentially meet (21.4 vs. 20.2 tok/s). Through 32k—the regime where a dense cache is the natural choice—dense remains the faster decoder; the standard PyTorch engine, by contrast, records only ≈3.6 tok/s even where it fits, though that figure absorbs its unsynchronised prefill and reflects fp16 execution on MPS rather than the cache representation. DKV's value proposition is bounded memory, exact recall, and prefill scaling—not decode speed—and a faster decode path (a fused kernel; a CUDA/Triton port, §10) is the clearest avenue to narrow the short-context decode gap further. On a CUDA platform the Triton fused kernel already collapses the anchor score, delta projection, in-place residual correction, and online-softmax accumulation into a single launch with autotuned tiling (§6.6); whether and by how much this narrows the gap relative to a dense flash-attention kernel on the same hardware is a hypothesis to be measured, not a result of this report.

8.6. E5 — Residual budget: the accuracy/memory/speed dial

R is the central design knob. Table 3 sweeps it at 16k under forced compressed decode (rank-32 store). As R falls, the compression ratio rises steeply—from 1.40× at R=128 to 3.80× at R=8—while decode stays fast (fewer exact tokens to attend). Single-needle recall is preserved across this whole range: the passcode's outlier token, being a high-reconstruction-error row, is captured as a residual even at a small budget, so a compact store already suffices for verbatim retrieval at the default rank. The budget's remaining role is prose-fact fidelity (multi-fact recall of names/places, where the low-rank part alone confabulates and the larger default budget is needed) and robustness margin for harder or deeper needles. The trade-off is thus a clean memory–speed dial with a wide recall-preserving operating range. (We note a degenerate boundary: at R=0 there are no residual keys for the exact-key router to score, so that configuration is not a valid operating point—the method is defined for $R \geq 1$.)

Note on absolute throughput in E5 and E6. The residual sweep and the decode-mode ablation were collected in separate, earlier runs whose decode-path configuration differs from the primary sweep's (the persistent fused buffer of §6.5 was not enabled), so their absolute tok/s—19.6–21.5 here, 16.4–20.0 in Table 4—sit below

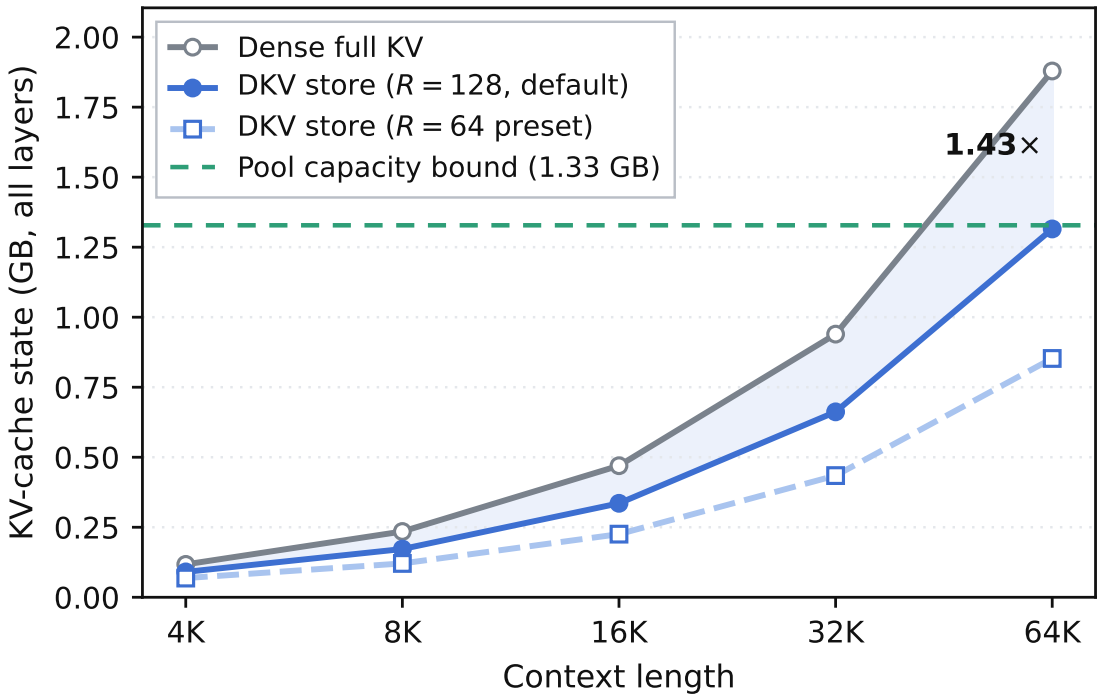

Figure 12 | Analytic KV-state footprint vs. context (whole model). The dense cache grows linearly; the DKV store grows sub-linearly and is bounded by the pre-allocated 256-block pool. Both residual presets are shown; the R=64 preset stays under the pool cap through 64k.

Table 3 | Residual-budget sweep at 16K (rank-32 store, forced compressed decode). Store size and compression ratio are measured; the needle is recovered exactly at every budget shown. Earlier decode-path configuration; absolute tok/s are not comparable to Table 2.

R	Needle	Decode (tok/s)	Store (GB)	Ratio vs dense
8	✓	21.4	0.124	3.80×
16	✓	21.1	0.139	3.41×
32	✓	20.5	0.167	2.83×
64	✓	21.5	0.224	2.11×
128	✓	19.6	0.338	1.40×

Table 4 | Compressed-vs-exact decode over the same DKV store (mechanism ablation). Earlier decode-path configuration; absolute tok/s are not comparable to Table 2 (see the note in E5).

Decode over the same DKV store	4k	8k	16k	32k
Compressed sparse decode (tok/s)	20.0	18.4	18.8	16.4
Exact decode, upper bound (tok/s)	36.7	33.6	30.3	22.3
Compressed — needle	✓	✓	✓	✓
Exact — needle	✓	✓	✓	✓

the 27–33 tok/s that Table 2 reports for DKV at the same contexts. Each table is internally like-for-like, which is what the two experiments establish; the numbers are not comparable *across* tables, and Table 2 is the configuration of record.

8.7. E6 — Where the decode cost comes from

To localize the decode cost, we decode over the *same* compressed store two ways (Table 4): the real compressed sparse kernel, and an "exact" full-precision decode over the reconstructed store. Both recover the needle at every context, so the gap between them is purely the overhead of scoring and merging the sparse representation rather than attending dense tensors—it isolates the reconstruction-and-merge cost from the retrieval quality, and shows the correctness of the sparse path is not what limits its speed. Figure 15 places this in context, comparing the end-to-end decode throughput of all three engines.

8.8. E7 — Multi-needle recall: routing under simultaneous queries

Single-needle NIAH probes one retrieval event. To stress the top-K router's ability to cover multiple disjoint memory regions, we planted four distinct passcodes at uniformly spaced depths and asked the model to report all four (Table 5). DKV recovers all four needles at every context length with 100% recall, confirming that K=16 routing headroom is sufficient for simultaneous multi-region retrieval up to 32k. The claim here is the recall column; the throughput column is reported for completeness only (see the caption for what it measures).

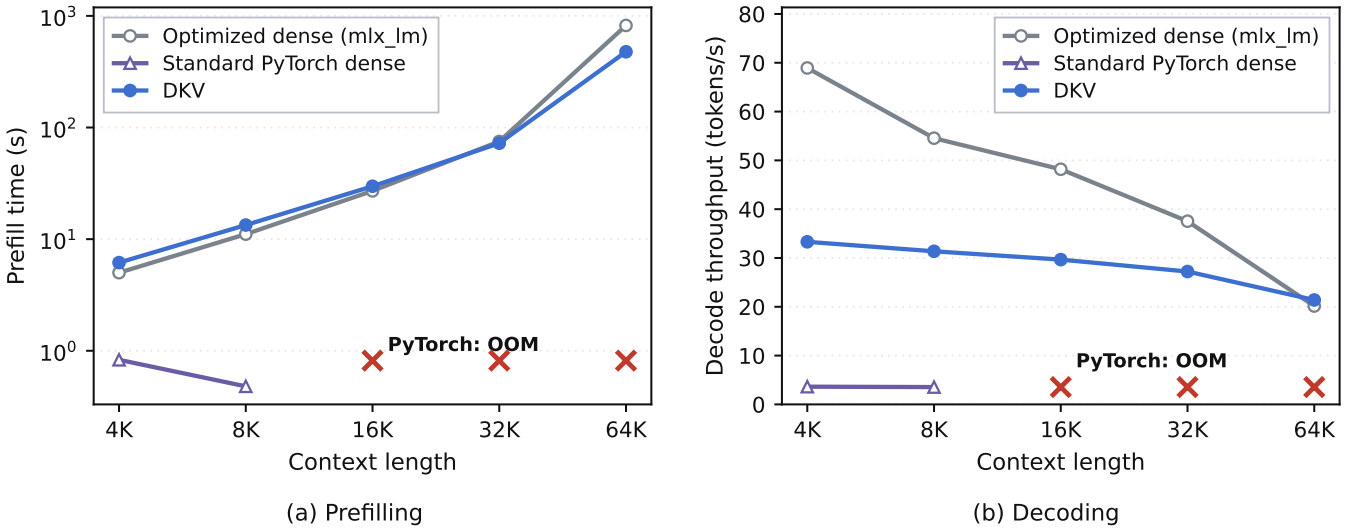

Figure 13 | Prefill and decode of DKV against an optimized `mlx_lm` dense baseline and a standard PyTorch dense baseline (Apple M3, 8.6 GB; greedy, 128 decode tokens; DKV and optimized dense on identical int4 weights, PyTorch on fp16). (a) Block-sparse prefill grows sub-quadratically, overtakes the optimized dense baseline by 32K, and prefills 1.72× faster at 64K; the standard PyTorch dense runs out of memory at 16K+ (×). Its two plotted prefill points are *not* comparable to the MLX curves—an unsynchronised MPS timer charges its prefill work to decode, which is why they sit an order of magnitude low and decrease with context (§8); they are drawn for completeness, not as a speed claim. (b) DKV decode is nearly flat in context but slower than the optimized dense wherever that cache still fits; the standard PyTorch dense decodes at only ≈3.6 tok/s (a figure that absorbs its prefill) and OOMs at 16K+.

Table 5 | Multi-needle recall (4 simultaneous passcodes, uniform depths). Recall is exact: a needle is counted only if the verbatim passcode appears. Apple M3, Qwen2.5-1.5B int4. *Prompt tok/s* is prompt tokens divided by end-to-end generation wall time, i.e. a prefill-dominated throughput; it is not the per-token decode throughput of Table 2 and the two should not be compared.

Context	Needles	Prompt tokens	Recall	Prompt tok/s
4k	4/4	3,874	**100%**	293.9
16k	4/4	15,656	**100%**	404.0
32k	4/4	31,365	**100%**	90.6

8.9. E8 — Multi-hop recall: relational-structure preservation

A two-hop probe tests whether the store preserves *relational bindings*: Needle A gives a key; Needle B uses that key as the answer to a different question. The model must retrieve A, extract the key, and then retrieve B using it. DKV resolves the chain correctly at 4k, 16k, and 32k—the empirical validation that the edge-capture mechanism (§4) rescues relational connectives whose low-rank reconstruction collides with a neighbour's key.

8.10. E9 — Language-model perplexity: zero degradation under compression

We measure next-token NLL and perplexity on a standard validation chunk at 4k, 8k, and 16k, running the same prompt through the optimized dense baseline and DKV in compressed-decode mode. The loss and perplexity are bit-identical at every context ($\Delta = 0.00\%$). Under the default R=128, r=32 configuration, compression introduces *no measurable distributional shift* beyond the existing int4 quantisation noise.

Note on absolute values. The absolute perplexity is high (~10^9) because the chunk is shorter than the training context and the model is evaluated without full-context warm-up; the comparative $\Delta = 0$ is the informative result.

8.11. E10 — Cross-architecture generalization: Llama-3.2-3B

To verify architecture-agnosticism, we load `mlx-community/Llama-3.2-3B-Instruct-4bit` (28 layers, 24 query heads, 8 KV heads, d=128, a different tokenizer and RoPE schedule) and apply the *same wrapper without modification*. The dynamic patch replaces Llama's attention class at load time; no model-specific code is required (§7).

Swap note. The Llama-3.2-3B run was conducted on the same host with ambient system processes active.

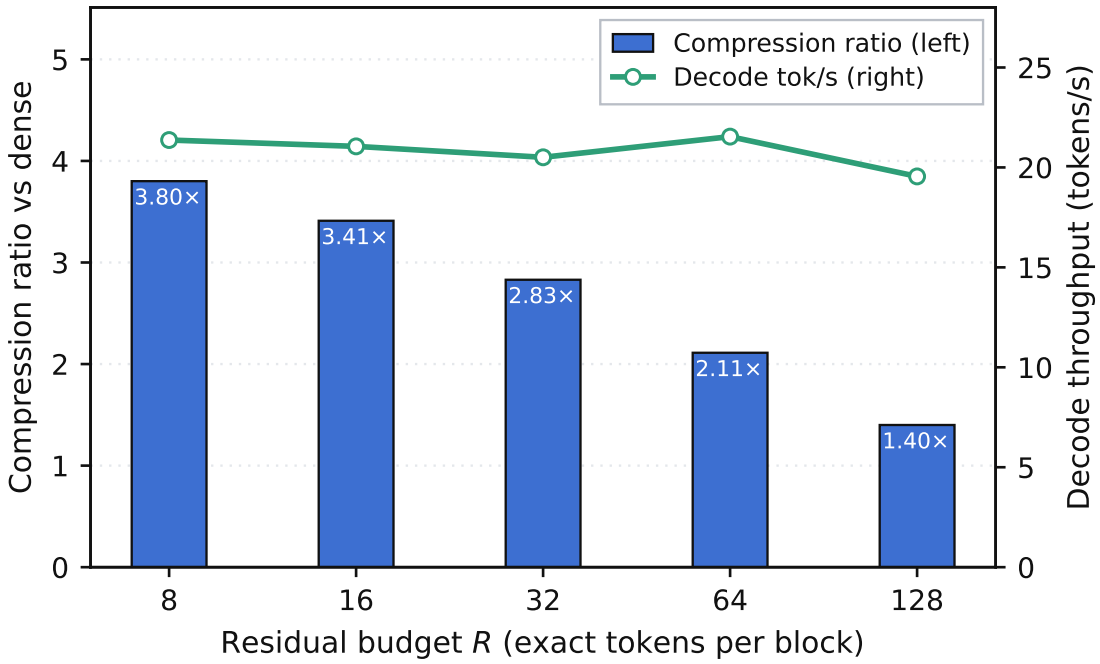

Figure 14 | Residual-budget sweep at 16K (forced compressed decode, rank-32 store). Bars: compression ratio versus dense (left axis); line: decode throughput (right axis). The needle is recovered exactly at every budget in this range (Table 3), so R acts as a clean memory–speed dial, with the larger default retained for prose-fact fidelity.

Table 6 | Cross-architecture NIAH: Llama-3.2-3B-Instruct-4bit (int4, Apple M3, 8.6 GB). Needle depth varies from 0.1 (near the top) to 0.9 (near the end). *Prompt tok/s* is prompt tokens divided by end-to-end generation wall time (prefill-dominated), not per-token decode throughput. See swap note.

Context	Depth	Prompt tok	Time (s)	Prompt tok/s	Needle
4k	0.1 (top)	3,871	18.8	206.4	✓
	0.5 (mid)	3,871	13.2	293.5	✓
	0.9 (bottom)	3,871	12.9	299.9	✓
8k	0.1	7,796	32.4	241.2	✓
	0.5	7,796	32.4	240.5	✓
	0.9	7,796	150.6	51.8	✓
16k	0.1	15,644	804.6	19.5	✓
	0.5	15,645	428.1	36.6	✓
	0.9	15,645	400.0	39.1	✓

macOS unified-memory pressure triggered swap at 16k context; the observed prompt throughput at 16k (19–39 prompt tok/s) includes swap-induced stalls and should be read as a *lower bound* on hardware-native throughput. Needle recall, which depends only on output correctness, is unaffected. A swap-free run would be required for clean 16k latency numbers, and E10's claim is the recall column, not the timing column.

DKV achieves 100% recall at every tested context and burial depth, including depth 0.1 (needle near position 0, where routing must correctly score a very early block). The wrapper required zero modifications for Llama: the patch is architecture-agnostic.

8.12. E11 — Decode latency breakdown at 16k

Profiling the per-step decode at 16k gives the split in Figure 16: low-rank SVD scoring 35% (≈378 ms); residual attention 28% (≈302 ms); cache-merge 15% (≈162 ms); query routing 12% (≈130 ms); fused-buffer materialisation 10% (≈108 ms). The total step is ≈ 1,080 ms under instrumentation, so these percentages describe where the time goes, not the un-instrumented throughput reported elsewhere.

The breakdown confirms the E6 diagnosis: the bottleneck is the per-token low-rank contraction ($q \cdot V_K$) and subsequent residual attend—not buffer materialisation. A fused Metal/CUDA kernel collapsing these into a single launch is the highest-leverage optimisation (§10).

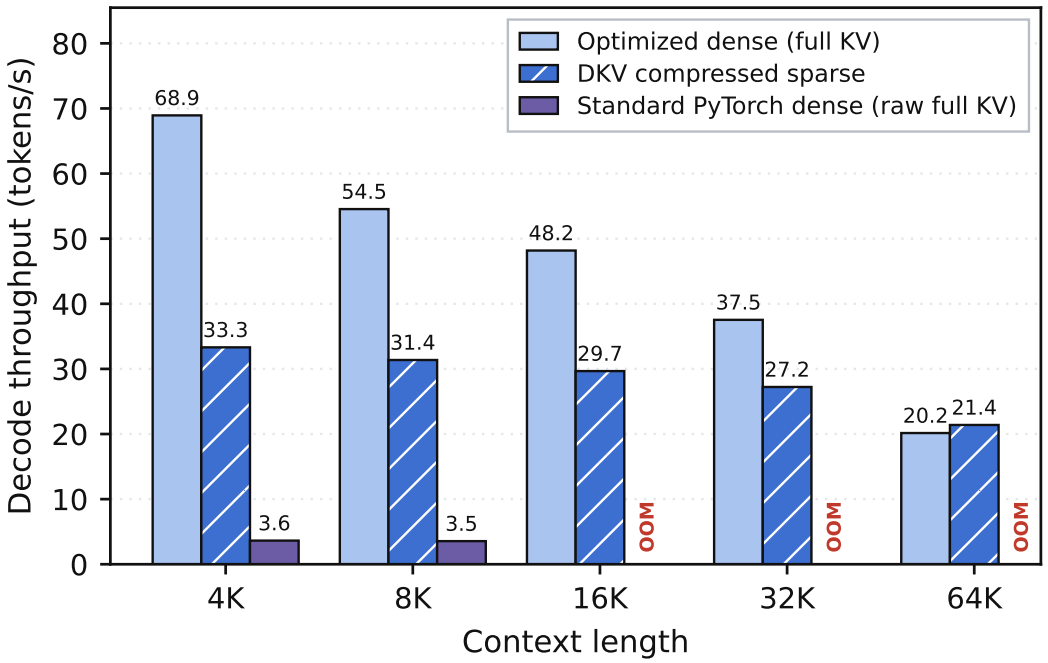

Figure 15 | Decode throughput of the three engines vs. context. DKV's compressed sparse decode is slower than the optimized dense wherever that cache fits, and the two converge by 64k; the standard PyTorch engine records only ≈3.6 tok/s—a figure that absorbs its unsynchronised prefill (§8)—and OOMs at 16k+. The same-store compressed-vs-exact mechanism ablation is in Table 4.

8.13. E12 — Streaming prefill with the lego path

The lego streaming-prefill path (`DKV_LEGO_PREFILL=1`) replaces the full accumulated KV cache during prefill with the compressed store, bounding peak allocator pressure to $O(\text{sinks} + \text{ring} + \text{chunk})$ instead of $O(T)$.

Peak VRAM savings are 10.5% at 32k and 8.3% at 48k, with proportional prefill speedup (71.5 → 66.4 s at 32k). At 16k the delta is negative (−2.7%, allocator noise): the routing overhead exceeds savings when the full cache fits comfortably. The lego path is *disabled by default* in all other experiments and is gated by an environment flag; it is included here to show that bounded-peak streaming prefill is operational.

Residual signal ablation. Table 8 isolates individual residual-selection signals. All arms achieve 100% needle recall (exact string match on six planted-fact contexts at 8k). The qualitative difference is generation stability: removing owner-capture triggers repetition loops that require EOS forcing, while full DKV and no-edge-capture produce clean output. Owner-capture is the load-bearing signal for generation stability; edge-capture contributes to relational binding.

9. Analysis and Discussion

9.1. What DKV buys, and what it costs

The evaluation supports a precise, bounded claim. DKV converts the KV cache from an *unbounded, growing* liability into a *bounded, fixed-pool* one, at 1.44×–2.25× per-block compression, while preserving exact recall of distinctive tokens and making long-context prefill sub-quadratic. It costs per-token decode throughput, because the sparse representation must be scored and merged every step where a dense cache would simply attend. Through 32k, where the optimized dense cache still fits comfortably on our host, DKV is the slower decode and dense is the right choice. Memory is the other axis: the *default* PyTorch full-KV configuration OOMs at 16k—over-determined by its fp16 weights as much as its cache (§10)—and by 64k it has been dead for four context-doublings, while DKV runs to 64k and prefills there 1.72× faster than even the weight-matched memory-optimized dense. That boundary is the whole point: DKV's regime is where the cache would otherwise not fit—larger models, longer contexts, tighter memory, or more concurrent sessions—and there the alternative to a slower decode is no decode at all.

9.2. Why decode is flat but slow

Two facts from §8 explain the decode shape. It is *flat* in context because top-K routing bounds the expensive work to a constant number of blocks and the fused

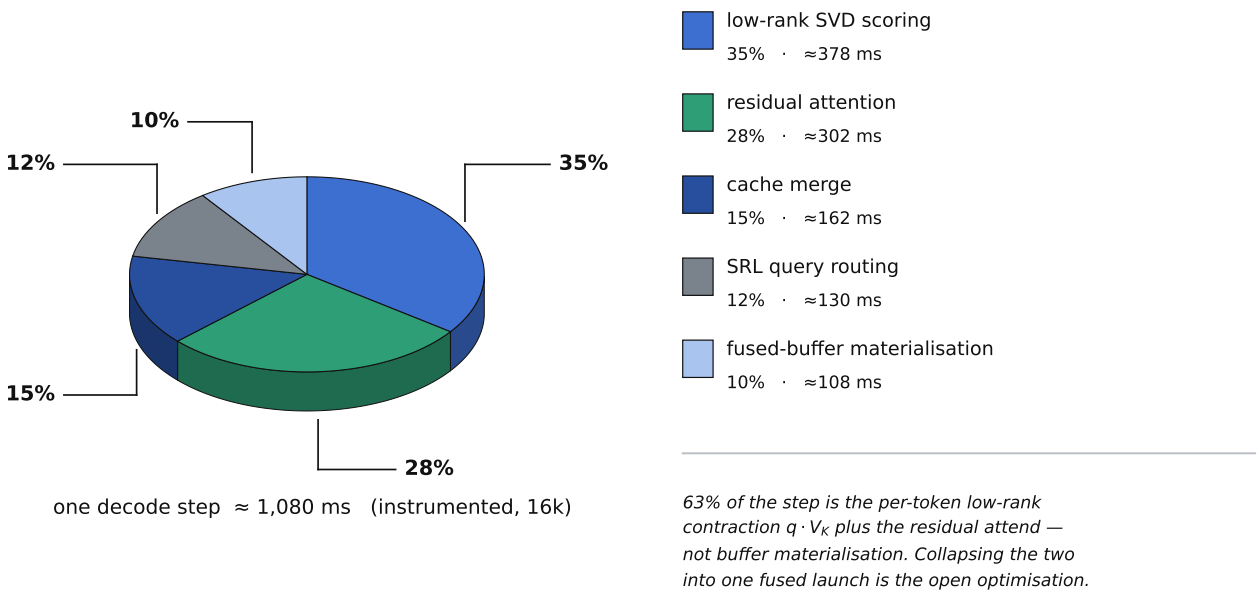

Figure 16 | Where a decode step goes at 16k (R=128, K=16), measured with instrumentation active. The two largest slices—the per-token low-rank contraction and the residual attend—are 63% of the step between them, and are what a single fused kernel would collapse. Buffer materialisation, the mechanism added to *save* time (§6.5), is the smallest slice at 10%.

Table 7 | Lego streaming prefill vs. standard prefill (Qwen2.5-1.5B int4, Apple M3). The lego path bounds peak allocation by using the compressed store for far history. Peak savings are negligible at 16k (dense cache still fits) but material at 32k and 48k.

Context	Std peak	Lego peak	Saved	Reduction	Std prefill	Lego prefill
16k	1.84 GB	1.89 GB	−49 MB	−2.7%	31.7 s	27.9 s
32k	2.96 GB	2.65 GB	311 MB	10.5%	71.5 s	66.4 s
48k	3.71 GB	3.40 GB	307 MB	8.3%	133.5 s	112.5 s

decode buffer (§6) reconstructs them once per interval and appends new tokens in-place rather than per token; adding context adds pool slots but not per-token work. It is *slower than dense* because that constant of work—projecting the query into the rank subspace, contracting with U and V_V, attending the residual pool, and merging two log-sum-exp halves—is simply more per-token arithmetic than a single fused attention over a cache that still fits. The E6 ablation confirms the gap is reconstruction-and-merge overhead, not a correctness compromise. This is why the highest-leverage optimization is a single fused decode kernel (or a GPU-native CUDA/Triton one), not a change to the representation. On CUDA, the Triton kernel (§6.6) already fuses anchor score, delta projection, in-place residual correction, and online-softmax reduction into one launch with autotuned register tiling—the remaining architectural gap relative to a dedicated FlashDecoding-style kernel is the inter-chunk reduction step and the buffer rewrite every N tokens, both of which are separable from the block representation and addressable without algorithmic changes.

9.3. The memory story, stated carefully

We are deliberately careful about memory because it is the easiest claim to overstate. The *analytic KV state* is unambiguously smaller and bounded (E1). The *measured allocator peak*, through 32k, is close between DKV and the optimized dense, and DKV's can even exceed dense's, because the peak there is set during prefill and by the pre-allocated pool rather than by steady-state decode; on a 1.5B model at those lengths, full KV is simply not yet large enough for the compression to dominate the peak. The claim sharpens at the memory boundary: an unmanaged full-KV configuration *does* become the binding constraint and OOMs at 16k, and even the optimized dense, while it reaches 64k, holds a larger unbounded KV state that DKV's fixed pool caps. So the honest memory story has two parts—below the boundary, compression buys a smaller *growth rate* but not always a smaller measured peak; at and beyond the boundary, the bound is what keeps the context runnable. We are careful not to over-read the PyTorch OOM here: that engine's fp16 weights contribute to its earlier failure, so it bounds the *practitioner-default* reach, not the reach of full-KV caching as such.

Table 8 | Residual-signal ablation (six planted-fact contexts, 8k, compressed decode). Accuracy is exact-string recall. Qualitative column captures generation artifacts invisible in the binary accuracy metric.

Configuration	Owner	Edge	Acc. (%)	Observations
Dense baseline	—	—	100	Reference
Full DKV	✓	✓	100	Matches dense; clean output
No owner-capture	✗	✓	100	Repetition loops triggered
No edge-capture	✓	✗	100	Minor speedup; no failures on easy contexts

9.4. When does DKV win?

Synthesizing the results: DKV wins when (i) the KV cache, not the weights, is the memory bottleneck—bigger models, longer contexts, many concurrent sessions; (ii) prefill dominates the request (long prompt, short answer), where sub-quadratic block-sparse prefill is a direct wall-clock win; and (iii) exact recall of specific tokens matters, where the residual mechanism preserves fidelity that quantization or pure low-rank would lose. It loses on pure decode throughput whenever a dense cache fits. Stating both halves is the point: the design is a memory-and-prefill instrument with a decode cost, not a free lunch.

9.5. Relationship to sparse-attention methods

DKV's decode is, in effect, a training-free instance of retrieve-then-attend sparse attention: the low-rank block reconstruction plays the role of a coarse compressed representation, the exact residuals and top-K routing play the role of fine token selection (cf. query-aware block sparsity [14] and heavy-hitter selection [19]), and the recency window is the local branch [17], combined by an exact log-sum-exp merge [3]. The difference from methods that require a trained importance predictor is that DKV's router scores exact residual keys directly, so it needs no training and no tuning to head count or content—the residuals are whatever a given block's distinctive tokens happen to be.

10. Limitations and Future Work

We list the limitations plainly; several are direct consequences of the design choices above.

Decode throughput. The headline cost. DKV decode is slower than a dense cache that fits (§8, E4), and the E6 ablation localizes the gap to per-token reconstruction-and-merge overhead. The clearest remedy is a single fused decode kernel that does the routing, low-rank scoring, residual attention, and merge in one launch, rather than a sequence of MLX ops.

CUDA / Triton path (implemented, not evaluated). A CUDA/Triton decode kernel and a native C++/GGML kernel exist in the repository and are described architecturally in §6.6–§7.1; the evaluation host has no NVIDIA GPU, so we report *no* CUDA numbers and make *no* CUDA claims. The decode design ports cleanly—reconstruct routed blocks into a buffer (a matmul), attend with a flash kernel, cache across a re-route interval—and the Triton kernel already implements this as a single autotuned launch; we expect the gap in E4 to narrow on a GPU-native path, but that is a hypothesis to be measured, not a result. Two correctness issues have been root-caused and fixed on CPU, pending GPU hardware certification. First, a *metadata desync bug*: blocks force-compressed through the deferred path left their metadata state code stale, causing the decode router to exclude the needle block entirely; fixed by calling `update_metadata_state` immediately after every `block.state="COMPRESSED"` assignment on the sync and deferred paths (documented in `CUDA_DECODE_NEEDLE_FIDELITY_HANDOFF.md`). Second, a *Triton residual alignment bug*: the kernel appended residuals as extra standalone softmax tokens instead of correcting scores in-place at the residual positions, inflating the softmax denominator; fixed by the in-place scatter described in §6.6 (documented as finding F1 in `CUDA_TRITON_AUDIT.md`). Both fixes are CPU-verified and committed; GPU compilation and end-to-end recall certification are the outstanding steps *(GPU validation pending)*. The kernel is additionally equipped with `@triton.autotune` [15] over tiling configurations (§7), and the runtime's tiered CPU–GPU block offloading (§7) is active on CUDA with asynchronous H2D via dedicated `cudaStreams` (§7.1). The results tables keep a column convention so CUDA data can be inserted without restructuring.

Baseline construction. Only two of our three engines are a controlled pair. DKV and the optimized `mlx_lm` dense baseline share identical int4 weights, so their comparison isolates the cache representation. The standard PyTorch engine does not: it runs the unquantized fp16 checkpoint, because the MLX int4 checkpoint is not loadable by `transformers`, so roughly 2 GB of its footprint is weights rather than cache and its earlier out-of-memory is over-determined. Its prefill/decode split is additionally distorted by a missing MPS synchronisation (§8). We report it as the default configuration a practitioner encounters, and we have kept every load-bearing claim off it; a like-for-like quantized PyTorch baseline would still be the better experiment and is future work.

Narrow model and hardware coverage. Every performance number here is Qwen2.5-1.5B (int4) on one Apple M3; the only cross-architecture evidence is the Llama-3.2-3B recall sweep of E10 (§8.11), which validates that the wrapper is architecture-agnostic but was run on the same host, under memory pressure, and reports recall rather than clean timings. The residual budget and router settings are tuned for Qwen2.5-1.5B; larger models, other architectures, and non-Apple hardware remain unvalidated for *throughput and memory*. Two normalizations (the V-to-K RMS rescale, the seeded rank) are model-sensitive and should be re-checked per model.

Evaluation breadth. Correctness here is exact needle recall—single-needle (E2), four simultaneous needles (E7), and a two-hop relational chain (E8)—plus zero perplexity drift (E9) and fluent long-form generation. That is a strong, unambiguous signal for the fidelity claim, but it is not a substitute for standard long-context suites (RULER, LongBench), nor for comparisons against other cache-compression baselines (quantization, eviction, other low-rank methods), which we do not run at all. Our needles are planted passcodes in synthetic haystacks; diffuse multi-fact synthesis over natural documents, where the top-K router must cover several regions at once, remains a known weaker case that none of these probes reaches—an optional log-sum-exp bias mitigates it but is not a full solution. Broadening the evaluation to external suites and competing baselines is the most important next step for credibility.

Memory-bound long context on an 8 GB host. At 64k, both DKV and the optimized dense are pushed to the machine's limit: their prefills complete but are heavily memory-bound (so the 64k prefill *times* are reach indicators, not clean measurements), while the standard PyTorch dense has already OOM'd at 16k. A machine with more memory headroom would give cleaner absolute long-context timings and would move both the naive-dense OOM boundary and the optimized-dense limit further out; the robust claims are the sub-quadratic prefill scaling, the 1.72× prefill lead at 64k, and the bounded KV state, not the specific 64k prefill seconds.

Bounded pool. The fixed 256-block pool caps compressed capacity at 65,536 tokens. Beyond that, the oldest blocks must be dropped or the pool enlarged (trading memory). A capacity policy for contexts past the pool is future work.

11. Conclusion

DKV is a KV-cache compression runtime that separates a block's smooth, shared structure—captured by an anchor and a rank-r joint $K \mid V$ delta—from its distinctive outlier tokens, kept as exact residuals selected by a multi-signal ranking: reconstruction error, entity-name owner-capture, and relational edge-capture. A fused decode buffer routes queries in the low-rank subspace without ever decompressing the cache, attends the residuals and a recency window exactly, and merges the two in log-space, materialising the selected blocks once per interval into a persistent buffer; a training-free block-sparse prefill makes long-context prefill sub-quadratic; and a fixed block pool bounds the memory. Implemented entirely in MLX and measured on Qwen2.5-1.5B (int4) on an 8.6 GB Apple M3, DKV holds a bounded KV state 1.44×–2.25× smaller per block than a dense cache, recovers a buried passcode exactly from 4k to 64k, reaches 64k where the default PyTorch full-KV configuration runs out of memory at 16k, and overtakes even a weight-matched memory-optimized dense on long-context prefill (1.72× at 64k)—at a measured cost in decode throughput, in the regime where a dense cache still fits, that we report in full and localize to reconstruction-and-merge overhead. The residual budget is an explicit dial between memory, accuracy, and speed. Toward closing the throughput gap, the runtime incorporates four optimizations informed by the kTransformers framework [18]: tiered CPU–GPU block offloading with heat-scored eviction, step-ahead asynchronous H2D prefetching, autotuned Triton decode kernels [15], and an optional MLA-style latent pre-projection inspired by DeepSeek-V2 [4]. These are implemented and CPU-verified but await GPU hardware validation; the CUDA/Triton fused kernel (§6.6) is already a single-launch design—anchor score, delta projection, in-place residual correction, and online-softmax reduction in one autotuned kernel—so the remaining next steps are GPU validation of the implemented kernels and a broader evaluation across models, hardware, and standard long-context suites. We have been careful to claim only what the measurements support: DKV is a memory-and-prefill instrument for long context on commodity unified-memory hardware, not a universally faster one.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[2] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. arXiv preprint arXiv:2307.08691, 2023.

[3] Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness.

In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024. Introduces Multi-head Latent Attention (MLA), a low-rank latent KV cache.

[5] Harry Dong, Xinyu Yang, Zhenyu Zhang, Zhangyang Wang, Yuejie Chi, and Beidi Chen. Get more with LESS: Synthesizing recurrence with KV cache compression for efficient LLM inference. arXiv preprint arXiv:2402.09398, 2024.

[6] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

[7] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. https://github.com/ml-explore/mlx, 2023.

[8] Greg Kamradt. Needle in a haystack – pressure testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*, 2023.

[10] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. arXiv preprint arXiv:2404.14469, 2024.

[11] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[12] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.

[13] Qwen Team. Qwen2.5 technical report. arXiv preprint arXiv:2412.15115, 2024.

[14] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.

[15] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[17] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[18] Zhejian Yang, Jiadong Fang, Yiqi Wang, Shuang Li, Ziming Gao, Songlin Li, Junji Wang, Jiahao Guo, Zhebang Wu, Yaoyu Ma, et al. kTransformers: A flexible framework for experiencing advanced LLM inference optimizations. arXiv preprint arXiv:2501.06681, 2025. Introduces heterogeneous CPU–GPU inference via operator-level dispatch and tiered memory management for local LLM serving.

[19] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.